RODRIGO BARBOSA DE SANTIS
TIAGO SILVEIRA GONTIJO

1a Edição

MACHINE LEARNING E PYTHON

Um guia prático à ciência de dados

Machine Learning e Python: Um Guia Prático à Ciência de Dados

Rodrigo Barbosa de Santis
Tiago Silveira Gontijo

1a Edição

Ficha Catalográfica

B238m Barbosa de Santis, Rodrigo e Silveira Gontijo, Tiago.

2023.

Machine Learning e Python: um guia prático à ciência de dados / Rodrigo Barbosa de Santis, Tiago Silveira Gontijo – Belo Horizonte: Editor: Tiago Silveira Gontijo. 1ª Edição, 2023.

61 p.

ISBN:

1. Ciência de Dados 2. Programação

3. Python

CDD: 620
CDD: 62

Prefácio

O aprendizado de máquina tem sido uma das áreas mais promissoras e em rápida evolução da ciência da computação nos últimos anos. Com a crescente quantidade de dados disponíveis e o aumento do poder de processamento dos computadores, a aplicação do aprendizado de máquina tem se tornado cada vez mais acessível e ampla.

Nesse contexto, o Python tem sido uma das linguagens mais populares para o desenvolvimento de projetos de aprendizado de máquina. Com sua sintaxe clara e intuitiva, vasta biblioteca de módulos e comunidade ativa, Python oferece uma ótima plataforma para implementar e experimentar algoritmos de aprendizado de máquina.

Este livro é destinado a todos os interessados em aprender sobre aprendizado de máquina e Python, desde iniciantes que desejam ter uma introdução sólida até aqueles que já têm algum conhecimento e buscam aprofundar seus estudos. Com uma abordagem didática e prática, este livro apresenta os conceitos fundamentais do aprendizado de máquina e como implementá-los em Python.

Ao longo deste livro, você será introduzido a diversos algoritmos de aprendizado de máquina, desde os mais simples até os mais complexos, e aprenderá a aplicá-los a problemas do mundo real. Além disso, você irá entender como preparar e processar dados, como avaliar o desempenho de seus modelos e como ajustá-los para obter melhores resultados.

Esperamos que este livro o ajude a se tornar um programador mais habilidoso e confiante em aprendizado de máquina com Python e, mais importante ainda, que você possa utilizar essas habilidades para resolver problemas desafiadores e impactar positivamente o mundo ao seu redor.

Conteúdo

1 Introdução **5**

2 Comandos básicos em Python **8**
2.1 Entrada e saída 8
2.2 Listas 9
2.2.1 Funcionalidades adicionais 10
2.3 Strings 11
2.4 Estruturas de controle e repetição 12
2.5 Métodos e funções 15
2.6 Classes 16
2.7 Dicionários 17
2.8 Exercícios 17

3 NumPy **19**
3.1 Array 20
3.2 Inicialização de arrays 21
3.3 Operações básicas 22
3.4 Funções e métodos 23
3.5 Exercícios 24

4 Análise exploratória de dados **27**
4.1 Leitura de arquivos 28
4.2 Gráficos 30
4.3 Exercícios 35

5 Aprendizado supervisionado **39**
5.1 Exemplo sintético 39
5.2 Separação dos conjuntos de treino e teste 40
5.3 O modelo linear 41
5.4 Extendendo o modelo linear 43
5.5 Testando outros classificadores 44

5.6 Ajuste de parâmetros 49
5.7 Métricas de avaliação de modelos 50
5.8 Exercícios 52

6 Aprendizado não-supervisionado **53**
6.1 Análise de Componentes Principais 54

Capítulo 1

Introdução

Python é uma linguagem de programação de alto nível que vem ganhando destaque na última década, devido seu amplo campo de aplicação e sua acessibilidade. Embora o nome da linguagem seja muitas vezes associado às serpentes da espécie Píton, a verdadeira história por trás do nome está relacionada a uma série britânica de muito sucesso nos anos 70, chamada "Monty Python's Flying Circus". O criador Guido Van Rossum escolheu este nome para o título de seu novo *hobby* pessoal, para que fosse um nome apelativo para *hackers* de UNIX/C.

A linguagem sempre teve o foco em ser muito simples, com o intuito de se fazer mais, com menos linhas de comando, possibilitando que o programador tenha controle, fácil interpretação dos seus códigos e manutenibilidade. A filosofia da linguagem é definida em um documento divulgado pelos próprios desenvolvedores, chamado de *PEP 20 – Zen of Python*. A seguir, apresentamos um excerto do documento:

> "Bonito é melhor do que feio
> Explícito é melhor do que implícito
> Simples é melhor do que complexo
> Complexo é melhor do que complicado
> Legibilidade faz diferença
> (...) "

Esta ideia de simplicidade e estética, aplicada em um ambiente tão complexo como o de desenvolvimento de software, vem atraindo novos programadores que compartilham destes valores e se entusiasmam-se com a linguagem, a ponto de que existem pessoas que a usam até para escrever poemas.

De volta ao ponto de vista prático, um dos pontos que mais contribuem com a popularização da linguagem, é o fato de ela ser gratuita e *open source*, sendo hoje a segunda linguagem mais popular em projetos disponibilizados no repositório GitHub - atrás apenas da linguagem web JavaScript.

Por se tratar de uma linguagem interpretada, a linguagem não possui arquivos binários compilados para proteção do código-fonte. Em alguns casos pode até ser criado um arquivo do tipo ".pyc" do tipo compilado, mas estes podem ser facilmente revertidos para o script original com a ajuda de alguns módulos. Como a linguagem têm forte aplicação em softwares comerciais, para garantir a proteção do código o que muitos desenvolvedores encontraram é através da biblioteca **Cython**, que transforma o código na linguagem C e realiza a compilação em arquivos binários. Desta forma, o código fica "blindado"para leitura dos usuários ou profissionais mal-intencionados.

Dentre das diversas áreas de aplicação da linguagem Python, destacamos as seguintes, e seus módulos mais conhecidos:

1. Desenvolvimento web e internet – com suporte a recursos diversos do ecossistema web, como html, xml, json, protocolos de internet; a linguagem é amplamente utilizada para desenvolvimento de projetos web e na internet. Um exemplo é o Dropbox, a solução de arquivos em nuvem que muito de nós usamos. **Django** e **Flask** são exemplos de frameworks disponibilizados neste segmento.

2. Científica e numérica – para cálculo numérico e operações vetoriais, a biblioteca **NumPy** é geralmente a mais conhecida, muito eficiente e otimizada. SciPy é outra biblioteca importante para estatísticas, contanto com diversos testes estatísticos paramétricos e não paramétricos, e manipulação de distribuições discretas e contínuas.

3. Aplicações *desktop* – desde o desenvolvimento de jogos (como por exemplo Civilization IV, Vega Strike) com a capacidade de processamento de gráficos 3D do **PySoy** e funcionalidades do **PyGame**, até o desenvolvimento de aplicações de com interfaces gráficas para usuário através das bibliotecas **PyQt** e **PyGtk**.

4. Inteligência computacional - Muitas das soluções utilizadas no nosso dia-a-dia em nossos celulares, televisores, etc, são desenvolvidas usando a linguagem, que é uma das principais linguagens utilizadas pela multinacional Google. O próprio módulo de deep learning **TensorFlow** desenvolvido e disponibilizado pela companhia faz uso da linguagem, que também conta com outras importantes bibliotecas como o **scikit-learn** e **Keras**.

No Capítulo 2 iremos nos familiarizar com algumas das sintaxes e funcionalidades básicas da linguagem.

Instalação

Por padrão, a maioria dos sistemas operacionais UNIX possuem o python pré-instalado. Para verificar, basta abrir o terminal e entrar o comando `python -V`, e será impresso a versão instalada do software. Módulos e bibliotecas podem ser adicionadas usando o gerenciador de pacotes **pip**.

No entanto, por razões de praticidade e produtividade, recomendamos a todos os leitores a instalação da distribuição Anaconda (https://www.anaconda.com/). O anaconda é uma distribuição da linguagem própria para o meio científico, que conta não só com a última versão do python, mas também com uma série de pacotes e bibliotecas que iremos utilizar no curso, como **NumPy**, **Pandas**, **Matplotlib**, **scikit-learn**, etc.

A distro Anaconda também faz a instalação das duas IDEs mais utilizadas em projetos de software da linguagem: o **Spyder**, que é uma interface gráfica própria para programação de scripts, muito simular ao RStudio e Matlab, e o **Jupyter Notebook** que permite a criação de notebooks, misturando blocos de código e de texto (linguagem *markdown*).

Capítulo 2

Comandos básicos em Python

2.1 Entrada e saída

Como já é de prache, começamos nosso contato com o Python escrevendo um programa que imprime na tela do usuário a mensagem "Hello world!". Para isso é utilizado a função `print()`. Esta função é básica da linguagem, e requer apenas um argumento do tipo string.

```
>> print("Hello world!")
Hello world!
```

A função também é capaz de imprimir variáveis de todos os tipos, realizando a conversão para o tipo string, e formatando de acordo com duas formas possíveis: utilizando a função `format()`, ou utilizando o coletor % após a definição da *string*.

```
>> s1 = "Hello"
>> s2 = "world"
>> print("{} {}!".format(s1,s2))
Hello world!
```

No próximo exemplo iremos utilizar o coletor para que sejam impressos apenas 3 casas decimais da variável numérica do tipo float. Esta forma de impressão é análoga à linguagem C.

```
>> n = 1.252124215
>> print("%.3f" % n)
1.252
```

Usando a função `format()` a declaração é feita entre chaves, conforme o exemplo abaixo.

```
>> n = 1.252124215
>> print("{0:.3f}".format(n))
```

```
1.252
```

Outra função básica amplamente utilizada para a leitura de dados do usuário é a função `input()`. Esta função pausa o programa e recebe o valor digitado pelo usuário, que pode ser atribuido a uma variável ou função.

```
>> name = input("Qual seu nome?")
>> print(name)
Qual seu nome? Ex.: Alberto
Alberto
```

Uma forma ainda mais reduzida de imprimir os valores de uma variável é adicionando 'f' antes da string. Desta forma, basta adicionar o nome da variável entre colchetes, conforme mostrado a seguir.

```
>> print(f"Valor de n: {n}")
Valor de n: 1.252124215
```

2.2 Listas

Podemos dizer que e a unidade elementar do Python são as listas. Além de ser uma estrutura de dados interessante e dinâmica, ela é base para a criação de dicionários e até mesmo estruturas de controles, como veremos a frente.

Inicializar uma lista é extremamente simples, basta você declarar seus valores dentro de um colchetes, separados por vírgulas:

```
>> lista1 = [1,2,3,4,5]
>> lista1
[1, 2, 3, 4, 5]
```

Note que ao atribuir um valor a uma variável, esta variável não é impressa no console. Para imprimir o valor, sem usar a função `print`, basta digitar o nome da variável em uma linha única, como fizemos acima.

Outra propriedade interessante das listas, é que elas são capazes de armazenar dados de diferentes tipos.

```
>> lista2 = [36,"Miguel",True,15.5]
>> lista2
[36, "Miguel", True, 15.5]
```

Uma forma mais automatizada para se definir uma lista é a partir da função `range()`, cujos parâmetros de entrada são: primeiro valor, limite superior, incremento. Veja o exemplo:

```
>> lista3 = list(range(10,100,10))
```

```
>> lista3
[10, 20, 30, 40, 50, 60, 70, 80, 90]
```

O operador $+$, quando usado em listas, não realiza a soma dos elementos, mas sim a concatenação dos elementos.

```
>> lista1 + lista2
[1, 2, 3, 4, 5, 36, 'Joao', True, 15.5]
```

O operador $-$ faz a exclusão dos elementos. O operador $*$ combinado com um inteiro n, replica uma lista n vezes.

```
>> 3*lista1
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

Para manipular selecionar um elemento de uma lista pelo seu índice, basta adicionar entre colchetes a posição desejada (a partir de 0).

```
>> lista3[1]
20
```

É possível utilizar índices negativos, neste caso o índice é contado desde o final da lista, e não o começo.

```
>> lista3[-1]
90
```

Pode-se selecionar uma parte da lista utilizando um índice dinâmico `a:b`, conforme o exemplo a seguir.

```
>> lista3[2:5]
[30, 40, 50]
```

2.2.1 Funcionalidades adicionais

Dada a seguinte lista:

```
>> a = [1,2,3]
>> b = [4,5,6]
>> c = a+b
>> c
[1, 2, 3, 4, 5, 6]
```

Podemos utilizar o método `pop()` da classe `list`, para eliminar o elemento de índice n.

```
>> c.pop(2)
>> c
[1, 2, 4, 5, 6]
```

O método `reverse()` inverte a lista, que agora está organizada de trás pra frente.

```
>> c.reverse()
>> c
[6, 5, 4, 2, 1]
```

A função `append()` pode ser usada para adicionar um novo elemento ao final da lista.

```
c.append(15)
c
```

Ainda algumas outras funções básicas para listas que podem ser usadas são `len()` – que retorna o tamanho da lista, ou seja, quantos elementos compõem a lista, ou a função `sum()`, que soma todos os elementos de uma lista.

```
>> len(c)
6


>> sum(c)
33
```

Finalmente, o método `clear()` apaga todos os elementos da lista, deixando-na vazia.

```
>> c.clear()
>> c
[]
```

2.3 Strings

Em Python, as strings também podem ser compreendidas como um tipo específico de lista, composta por caracteres em uma sequência definida.

Dada a seguinte string:

```
>> s1 = 'Aprendizado de Maquina com Python'
>> print(s1)
Aprendizado de Maquina com Python
```

Utilizando o método `split()`, podemos transformar uma string em um vetor de strings, a partir de um separador definido – pode ser um carácter como ou conjunto de caracteres.

```
>> s1.split(' ')
['Aprendizado', 'de', 'Maquina', 'com', 'Python']
```

Para separarmos uma string em mais de um separador, pode-se utilizar o pacote **re**. Utiliza-se '|', que é conhecido em programação como o operador lógico OU. No caso do ponto, também é preciso colocar uma barra invertida '\', que é sempre utilizada em caracteres especiais, como exemplo '*' e '\n'.

```
>> email = "antonio@hotmail.com.br"
>> import re
>> re.split('@|\.',email)
('antonio', 'hotmail', 'com', 'br']
['Aprendizado', 'de', 'Maquina', 'com', 'Python']
```

Analogamente às listas, um índice pode ser usado para selecionar uma parte de uma string.

```
>> s1[:15]
'Aprendizado de '
```

Alguns métodos são bastante úteis para formatação de strings, são eles `upper()`, que transforma todas as letras para maiúsculo;`lower()`, que transforma as letras para minúsculo e `title()`, que mantém apenas o primeiro carácter de cada palavra em maiúsculo e os demais em minúsculo.

```
>> s1.upper()
'APRENDIZADO DE MAQUINA COM PYTHON'
```

O método `replace()` pode ser utilizado para substituir um carácter ou grupo de carácter por outro definido.

```
>> s1.replace('e','3')
'Apr3ndizado d3 Maquina com Python'
```

E o método `find()` retorna o índice em que um carácter ou grupo de carácter foi/foram encontrados.

```
>> s1.find('Python')
27
```

Utilizando a indexação, podemos imprimir a string a partir da palavra Python.

```
>> s1[s1.find('Python'):]
```

2.4 Estruturas de controle e repetição

Como havíamos comentado anteriormente, as listas são importantes nas estruturas de controle e repetição da linguagem. O loop `for` ocorre sempre em uma lista, assumindo o valor de cada um dos elementos desta lista.

Neste primeiro exemplos mostramos essa propriedade, em que a variável de controle i assume o valor de cada um dos elementos da lista de 0 a 5, com incremento 1.

Importante observar que nesta linguagem, as tabulações são importantes para definir quais comandos estão incluídos em um loop ou comparação.

```
>> for i in range(5):
>>     print(i)
0
1
2
3
4
```

Utilizando-se a função `enumerate()`, é possível associar um índice a uma lista conhecida, na ordem em que os elementos estão ordenados. Este recurso é útil quando além de se iterar por todos elementos de uma lista, precisamos de um índice de contagem para selecionar alguma outra variável necessária ao nosso problema.

```
>> abc = ['a','b','c','d']
>> for i, l in enumerate(abc):
>>     print(i,l)
0 a
1 b
2 c
3 d
```

Quando se quer iterar em duas listas simultaneamente, a função `zip()` pode ser utilizar, conforme demonstrado no exemplo a seguir.

```
>> xyz = ['x','y','z']
>> for a, x in zip(abc,xyz):
>>     print(a,x)
a x
b y
c z
```

Note que no entanto, a lista dupla criada tem o mesmo tamanho da menor das listas, e os elementos adicionais da lista maior serão ignorados.

A estrutura `while` também está presente na linguagem, esta não requerendo uma lista mas sim uma condição de igualdade para manter-se em operação.

```
>> i = 1
>> while i < 6:
>>     print(i)
>>     i += 1
1
2
```

```
3
4
5
```

É possível alterar os parâmetros da função de inicialização de listas `range()`, de acordo com a necessidade do programa.

```
>> for i in range(10,25,3):
>>     print(i)
10
13
16
19
22
```

O comando `if`, bem como `else` e `elif` (senão-se), também estão disponíveis na linguagem.

```
>> numero = int(input("Digite um numero: "))
>> if numero % 2 == 0:
>>     print("Par.")
>> else:
>>     print('Impar.')
Digite um numero: 16
Par.
```

O comando `in`, faz a busca de uma determinada variável em todos os elementos de uma lista, retornando `TRUE` caso encontrado e `FALSE` caso não encontrado.

```
>> frutas = ['Banana', 'Laranja', 'Morango']
>> if 'Morango' in frutas:
>>    print("Tem.")
>> else:
>>    print("Nao tem.")
Tem.


>> if 'Carro' in frutas:
>>     print("Tem.")
>> else:
>>     print("Nao tem.")
Nao tem.
```

Ainda que na linguagem não exista diretamente explícito o comando `switch`, ele pode ser simulado utilizando a estrutura `if`,`elif`,`else`.

```
>> nota = int(input("Entre a nota (0-100):"))
>> if nota < 60:
>>     print("F")
>> elif nota < 70:
```

```
>>     print("D")
>> elif nota < 80:
>>     print("C")
>> elif nota < 90:
>>     print("B")
>> elif nota <= 100:
>>     print("A")
>> else:
>>    print("Nota invalida.")
Entre a nota (0-100):85
B
```

A vantagem da utilização do `elif` é que ele só realiza a segunda comparação, se a primeira não for verdadeira, e assim por diante. Desta forma é uma boa opção para economizar recursos computacionais no código, colocando as condições encadeadas da mais genérica para a mais específica.

O `if` pode ser declarado em uma única linha, conforme mostrado abaixo. A principal vantagem é manter a estética e manutenabilidade do código, pois um comando `if-else` geralmente requer ao menos 4 linhas para execução.

```
>> clima = 'ensolarado'
>> sair = True if clima == 'ensolarado' else False
>> sair
True
```

2.5 Métodos e funções

Para definir métodos em função, usamos o comando *def*. Após sua declaração, a função estará disponível para uso. É importante tomar cuidado com a endentação, que é usada para delimitar os comandos dentro da função.

```
>> def sem_retorno():
>>     print("Este metodo nao retorna valores.")
>> sem_retorno()
Este metodo nao retorna valores.
```

Para passarmos parâmetros, basta definirmos seu nome dentre os parâmetros. Podem ser definidos diversos parâmetros para as funções. O comando *return* é usado para retornar as variáveis da função.

```
>> def media(lista):
>>     return sum(lista)/len(lista)
>> media([10,14,20])
14.666666666666666
```

Valores padrões podem ser definidos, neste caso se tornando opcionais. Parâmetros sem valores padrão, são considerados obrigatórios e a função não executa se este não for recebido.

```
>> def imprimir_ate(lista, n=5):
>>     print(lista[:n])
>> imprimir_ate(list(range(1,100)))
[1, 2, 3, 4, 5]
```

2.6 Classes

A linguagem Python ainda suporta programação orientada a objetos. Na linguagem orientada a objeto, define-se uma

Um exemplo é mostrado a seguir onde é declarada uma classe *Gato*. A única variável que esta classe possui é o nome, sempre fixo, com o valor "Garfield".

```
>> class Gato:
>>     nome = "Garfield"
>>
>> meuGato = Gato()
>> print(meuGato.nome)
Garfield
```

Um exemplo um pouco mais complexo, é a classe *Cachorro* a seguir. As instâncias dessa classe possuem duas propriedades, *nome* e *raca*; e dois métodos *__init__()* que é o método padrão de construção de classe e o método *latir()*, que imprime uma frase com o nome dado a instância do objeto.

> *__init__()* é um método oculto de todas as classes na linguagem Python. Ele é o método que é executado ao ser inicializada uma nova instância da classe.

```
>> class Cachorro:
>>     def __init__(self, nome, raca): # Construtor de classe
>>         self.nome = nome
>>         self.raca = raca
>>
>>     def latir(self):
>>         print("Woof! - exclamou {}.".format(self.nome))
>>
>> meuCachorro = Cachorro("Marley", 'Labrador')
>> meuCachorro.latir()
Woof! - exclamou Marley.
```

2.7 Dicionários

Dicionários é uma estrutura de dados disponível na linguagem conhecida como vetor associativo. Um dicionário consiste numa coleção de pares chave-valor. Para cada chave, um valor associado está mapeado.

Em aprendizado de máquina, esta estrutura de dados é utilizada em diversas aplicações, como por exemplo: registrar os resultados de um estimador, escolher parâmetros para modelos ou conjunto de parâmetros de busca para a seleção de modelos, definir um dicionário de mapeamento.

```
>> pessoa = {
>>     'nome': 'Hugo',
>>     'idade': 25,
>>     'peso': 73.5,
>>     'fumante': False,
>> }
>> pessoa
{'nome': 'Hugo', 'idade': 25, 'peso': 73.5, 'fumante': False}

>> pessoa['nome']
'Hugo

>> pessoa['idade'] = 35
{'nome': 'Hugo', 'idade': 35, 'peso': 73.5, 'fumante': False}
```

2.8 Exercícios

Os seguintes exercícios são propostos para fixação de conhecimento deste capítulo.

Exercício 1. Peça ao usuário para digitar uma string e imprima se esta string é um palíndromo ou não. (Palíndromo é uma palavra que é lida da esquerda para a direita, conforme o sentido habitual da leitura, ou da direita para esquerda).

Exercício 2. Escreva um programa que pergunte ao usuário quantos números de Fibonacci serão gerados. Aproveite esta oportunidade para pensar sobre como você pode usar funções. Certifique-se de pedir ao usuário para inserir o número de números na sequencia para gerar. (Dica: A sequencia Fibonacci é uma sequencia de números onde o próximo número na sequencia é a soma dos dois números anteriores na sequencia. é assim: 1, 1, 2, 3, 5, 8, 13,...).

Exercício 3. Forca é um jogo muito popular que todos provavelmente conhecem e já jogaram quando crianças. Neste jogo, um jogador define uma palavra

secreta, e o outro jogador tem um número limitado (normalmente 6) de letras máximas que ele pode errar. Cada vez que uma palavra certa é adivinhada, é mostrado em qual posição se encontram as letras. Crie um jogo de forca para utilizando a linguagem Python, tentando incluir o máximo de funcionalidades que achar pertinente.

Capítulo 3

NumPy

NumPy é uma biblioteca poderosa para cálculo numérico, amplamente difundida e utilizada por quase todas as outras bibliotecas existente. Ela foi construída de forma que os comandos as operações entre seus elementos sejam realizados em sub-rotinas otimizadas em C e C++. Graças a esta propriedade conseguimos utilizando uma linguagem de alto nível, ter um desempenho muito bom em nossos programas.

Para cientistas de dados/pesquisadores que fazem modelos de aprendizado em Python, é fundamental o conhecimento básico dessa biblioteca, visto que muita das matrizes que iremos passar como parâmetro para nossos estimadores, bem como os resultados obtidos, serão do tipo básico dessa biblioteca, o *Array*.

Nos dois programas abaixos mostramos tempo necessário para realizar uma multiplicação de matriz utilizando a linguagem python pura, comparada com o tempo . Veja como o tempo é relativamente menor.

```
# Python puro
>> import time
>> l = 10000000
>> start = time.time()
>>
>> a, b = range(l), range(l)
>> c = []
>> for i in a:
>>     c.append(a[i] * b[i])
>>
>> t = time.time() - start
>> print( " Tempo: %s" % t)
Tempo: 4.49s
```

```
# NumPy
>> import time
>> import numpy as np
>> l = 10000000
>> start = time.time()
>>
>> a = np.arange(l)
>> b = np.arange(l)
>> c = a * b
>>
>> t = time.time() - start
>> print( " Tempo: %s" % t)
Tempo: 0.37 s
```

3.1 Array

Arrays são vetores multi-dimensionais que suportam dados de um tipo único, definido em sua inicialização. No exemplo abaixo iremos declarar nosso primeiro vetor usando a função *array*, de forma bem parecida com o que fazíamos com as listas.

```
>> import numpy as np
>> a = np.array([0, 1, 2, 3, 4])
>> a
array([0, 1, 2, 3, 4])
```

Note que para utilizar *arrays* e as funcionalidades da biblioteca NumPy, é necessário importar o módulo usando o comando *import*. O comando *as* quando usado define um "apelido"para o módulo, e em geral as pessoas importam esta biblioteca apelidando ela de 'np', para deixar o código mais enxuto.

Ao declarar nosso *array*, o módulo varreu todos os elementos e já definiu um tipo para o objeto: neste caso por todos valores serem inteiros, ele usou o tipo 'int32'. O usuário pode fixar um tipo caso deseje, como veremos mais tarde.

```
>> a.dtype
dtype('int32')
```

Uma propriedade da classe array que geralmente consultamos muito quando estamos desenvolvendo um script, é a propriedade *shape*. Ela retorna um par (ou *tuple*, em inglês), com as dimensões do vetor. Nesse primeiro caso, temos um vetor de 1 única dimensão e tamanho 5.

```
>> a.shape
(5,)
```

Definiremos agora uma matriz, que é representado como um vetor multi-dimensional. Para isso são usadas listas dentro de listas, com os valores dos elementos.

```
>> b = np.array([[1.5, 2, 3], [5, 5, 6], [7, 8, 9]])
>> b
array([[1.5, 2. , 3. ],
       [5. , 5. , 6. ],
       [7. , 8. , 9. ]])
```

Como desta vez colocamos números com casas decimais, o programa criou um objeto com do tipo 'float64'. Este tipo ocupa mais espaço em memória do que o inteiro.

```
>> b.dtype
dtype('float64')
```

Finalmente, as dimensões do nosso novo vetor multidimensional é 3x3, e podemos verificar isso usando novamente a propriedade *shape*.

```
>> b.shape
(3, 3)
```

3.2 Inicialização de arrays

Veremos agora outras formas mais automáticas de inicializar um *array*. Uma das formas mais comuns de se inicializar é usando a função *arange()*, que funciona como um iterador, bem similar a função *range()* que usamos para listas.

```
>> a = np.arange(1,10)
>> a
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Uma outra função é o *linspace()*, que cria n elementos dentro de um intervalo definido. Esta função assim como algumas outras que veremos a seguir são bem semelhantes à linguagem Matlab.

```
>> a = np.linspace( 0, 2, 9)
>> a
array([0. , 0.25, 0.5 , 0.75, 1. , 1.25, 1.5 , 1.75, 2. ])
```

A função *zeros()* cria um vetor com apenas elementos nulos. No exemplo passamos como argumento da função o par ordenado (2,2), que é interpretado pela função como uma matriz 2x2.

```
b = np.zeros( (2,2) )
b
array([[0., 0.],
      [0., 0.]])
```

A função *ones()* cria um vetor com elementos 1, e a função *eye()* é utilizada pra criar uma matriz identidade. No exemplo a seguir criamos uma matriz 3x4, com elementos 1, e definimos o tipo do vetor como 'complex'.

```
>> b = np.ones( (3,4), dtype=complex)
>> b
array([[1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j],
      [1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j],
      [1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j]])
```

Após inicializado, um método que geralmente é bem usado é o *reshape()*, que altera a dimensão do vetor para um novo valor. No exemplo abaixo iremos transformar um vetor 1x10.000 em um vetor 100x100.

```
>> c = np.arange(10000)
>> c
array([  0,   1,   2, ..., 9997, 9998, 9999])

>> c.reshape(100,100)
array([[  0,   1,   2, ...,  97,  98,  99],
       [ 100, 101, 102, ..., 197, 198, 199],
       [ 200, 201, 202, ..., 297, 298, 299],
       ...,
       [9700, 9701, 9702, ..., 9797, 9798, 9799],
       [9800, 9801, 9802, ..., 9897, 9898, 9899],
       [9900, 9901, 9902, ..., 9997, 9998, 9999]])
```

3.3 Operações básicas

Diferentemente das listas, os operadores matemáticos em vetores realizam o cálculo entre o vetor e um escalar ou outro vetor. Note que em caso de operações entre vetores, é necessário que eles tenham a mesma dimensão. Os operadores '+' e '-' fazem a soma ou substração.

```
>> a = np.array([10, 20, 30, 40])
>> b = np.ones( 4 )
>> a + b
array([11., 21., 31., 41.])
```

Enquanto '*' e '/', fazem a multiplicação elemento a elemento.

```
>> a * 2
array([20, 40, 60, 80])
```

Usando o operador '**' é possível fazer o cálculo de potência:

```
>> a ** 2
array([ 100, 400, 900, 1600], dtype=int32)
```

Os operadores lógicos '<' (menor que), '>'(maior que), '==' (igual) e '!=' (diferente) estão disponíveis para vetores. Seu uso retorna em um vetor de elementos do tipo binário, ou *Boolean*.

```
>> a < 30
array([ True, True, False, False])
```

Operações lógicas também podem ser utilizadas para selecionar um sub-espaço do vetor, dada uma condição,

```
>> a[a < 30]
array([10, 20])
```

Diferente de outras linguagens de programação, o operador de multiplicações entre vetores '*' e '/' em python não realiza a multiplicação matricial, e sim a multiplicação elemento a elemento.

```
>> A = np.array( [[1,1],
>>               [0,1]] )
>> B = np.array( [[2,0],
>>              [3,4]] )
>> A * B
array([[2, 0],
      [0, 4]])
```

O operador para multiplicação matricial é o '@'. O mesmo resultado pode ser obtido utilizando o método *dot()* do objeto *array*.

```
>> A @ B
array([[5, 4],
      [3, 4]])

>> A.dot(B)
array([[5, 4],
      [3, 4]])
```

3.4 Funções e métodos

A biblioteca NumPy conta com diversas funções matemáticas, próprias para operação de vetores, tais como expoente, logaritmo, trigonométricas (seno, cosseno, tangente, etc.), dentre outras. A função *exp()* a seguir retorna a constante neperiana elevada a um valor passado como argumento.

```
>> np.exp(1)
2.718281828459045
```

Quando se utiliza uma função do módulo para um vetor, o retorno é um vetor de mesma dimensão. No exemplo a seguir *sqrt()* retorna a raíz quadrada de cada elemento.

```
>> np.sqrt([16,25,36])
array([4., 5., 6.])
```

O sub comando

```
>> s = np.random.random( (3,5) )
>> s
array([[0.05272236, 0.36741613, 0.73996459, 0.2594056 , 0.22956865],
      [0.25507074, 0.46589706, 0.01894638, 0.67840388, 0.10201618],
```

```
[0.84788065, 0.02326221, 0.89858739, 0.92990781, 0.66036056]])
```

Existem ainda uma série de métodos que trazem informações descritivas básicas dos vetores, são eles *mean* (média), *std* (desvio-padrão), *var* (variância), *min* (mínimo), *max* (máximo), *sum* (soma).

```
>> s.std()
0.32082472825290304
```

Por fim, usando o argumento axis, o usuário pode definir se as funções serão calculadas por linhas (axis=1), por coluna(axis=1), ou por todos os elementos (axis=None, valor *default*).

```
>> s.mean(axis=1)
array([0.32981547, 0.30406685, 0.67199972])
```

3.5 Exercícios

Exercício 1. Uma das métricas mais utilizadas em problemas de regressão é o erro absoluto percentual médio – MAPE, ou *Mean Absolute Percentage Error*). Esta métrica não está disponível no pacote *scikit-learn*, mas você precisa desta medida para poder comparar seus modelos de regressão baseados em aprendizado de máquina com os modelos estatísticos atualmente aplicados em sua empresa.

Desenvolva uma função que seja capaz de retornar esta métrica para você.

A fórmula do MAPE é dada por:

$$MAPE = \frac{1}{n}\sum_{i=1}^{n}\frac{|Y_i - \hat{Y}_i|^2}{Y_i} \tag{3.1}$$

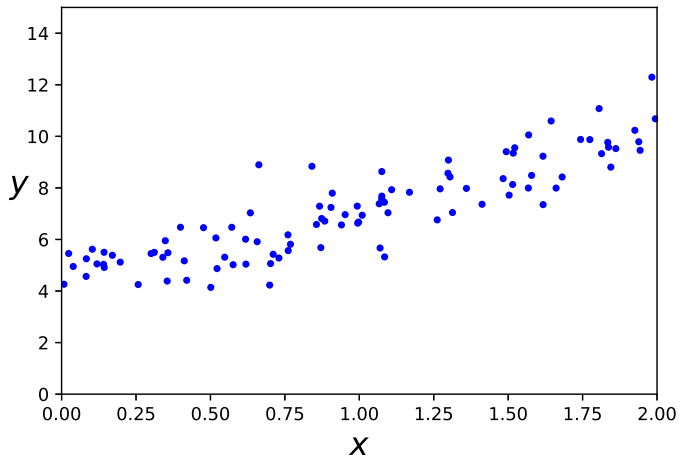
onde n é o número de observações, Y_i o valor observado e $\hat{Y}_i$ o valor previsto.

Observar o tratamento de exceções quando Y_i for zero ou muito próximo de zero, pois pode resultar em problemas na divisão.

Exercício 2. Dada a amostra aleatória Y, criada em função da variável aleatória X, gerada a partir do código abaixo.

```
>> X = 2 * np.random.rand(100,1)
>> y = 4 + 3 * X + np.random.randn(100,1)
>>
>> plt.plot(X, y, 'b.')
>> plt.xlabel("x", fontsize=18)
>> plt.ylabel("y", rotation=0, fontsize=18)
```

```
>> plt.axis([0, 2, 0, 15])
```



Gostariamos de ajustar um modelo linear a estes dados, tal que a saída do estimador $h_\theta(x_i)$ é dada por:

$$h_\theta(x_i) = \theta_0 + \theta_1 x_i \tag{3.2}$$

Nossa função de erro J (ou função custo) é dado pelo erro quadrático médio das nossas previsões e o valor observado, conforme a equação abaixo.

$$J(\theta) = \frac{1}{M}\sum_{i}^{M}(h_\theta(x_i) - y_i)^2 \tag{3.3}$$

No algoritmo do gradiente descendente, iremos atualizar os valores de θ, utilizando-se um incremento de tamanho α, de tal forma que o nosso erro total diminua a cada iteração.

$$\theta_i^{(1)} = \theta_i^{(0)} - \alpha\frac{\partial}{\partial\theta_i}J(\theta_i) \tag{3.4}$$

Mais especificamente para o modelo de regressão linear, a derivada da função do erro J pelos coeficientes θ_i é dado por:

$$\theta_0^{(1)} = \theta_0^{(0)} - \alpha\frac{1}{m}\sum_{1=1}^{M}(h_\theta(x_i) - y_i) \tag{3.5}$$

$$\theta_1^{(1)} = \theta_1^{(0)} - \alpha\frac{1}{m}\sum_{1=1}^{M}(h_\theta(x_i) - y_i)x_i \tag{3.6}$$

Utilizando as duas equações acima e o código disponível no repositório GitHub da disciplina, implemente o algoritmo de gradiente descendente. O que acontece quando a taxa de aprendizado α e o número de iterações são alterados? Extra: plote um gráfico com a evolução do erro J a cada iteração e veja como este muda com alteração dos parâmetros.

Exercício 3. O algoritmo de gradiente descendente também pode ser adaptado para problemas de classificação. Neste tipo de problema, nossa variável preditiva é binária. O modelo estatístico mais conhecido para este tipo de aplicação é a regressão logística.

Trata-se de uma regressão linear com uma função de ativação $\sigma(z)$ ou 'sigmóide', que transforma a saída da regressão linear em uma saída entre 0 e 1, constantemente associado a uma probabilidade. A saída do modelo é dada por

$$h_\theta(x_i) = \sigma(\theta_0 + \theta_1 x_i) \tag{3.7}$$

tal que, $\sigma(z)$ é expresso por:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{3.8}$$

Para esta saída se tornar discreta, definimos um ponto limítrofe (normalmente $0,5$) tal que, se a probabilidade de saída do modelo é maior do que o limite estimado, a classe predita é a classe 1, e se menor, a classe predita é a classe 0.

A função custo J desta função, é um pouco diferente do modelo linear, e é conhecido como função de perda (loss function), expresso por:

$$J(\theta) = \frac{1}{M}(-Y^T \log(h) - (1 - Y)^T \log(1 - h(X))) \tag{3.9}$$

E para este caso, a função de atualização fica:

$$\theta^{(1)} = \theta^{(0)} - \frac{1}{M}.\alpha.X^T(h(X) - Y) \tag{3.10}$$

Usando a última equação e o script disponível no GitHub, atualize a função de forma a ajustar o modelo logístico ao conjunto de dados Iris.

Capítulo 4

Análise exploratória de dados

Antes de ajustar qualquer modelo de previsão, sela ele estatístico ou baseado em aprendizado de máquina, uma habilidade fundamental para qualquer cientista de dados é a capacidade de entender o domínio da aplicação que está sendo estudada e o modelo gerador dos dados.

Cada tipo de variável ou conjunto de variáveis possui um tipo de representação gráfica mais apropriada, dependendo do contexto. Por exemplo, em séries temporais, a forma mais comum de representar é a partir de um gráfico de linhas, com o eixo x representando o tempo. No mercado financeira, por exemplo, é comum encontrar gráficos *candlestick* para representar a variação de preços de um determinado ativo.

Embora não exista fórmula mágica para determinar a melhor representação de uma variável de interesse, conseguimos sumarizar algumas práticas mais comuns para as variáveis de acordo com sua natureza.

Variáveis **categóricas** geralmente são representadas a partir de **gráficos de contagem**, enquanto para variáveis **numéricas** usamos **histogramas**. Para exprimir as relações entre variáveis numéricas podem ser aplicados **gráficos de dispersão**, **conjuntos** ou **de pares**, enquanto para relações entre variáveis numéricas e categóricas usam-se **gráficos de caixa** (mais popularmente conhecidos como *boxplots*) ou gráficos condicionais.

Em Python contamos com algumas ferramentas excelentes de análise exploratória de dados. **Pandas** é uma biblioteca voltada a manipulação de *data frames*, que são tabelas de dados dispostas de forma estruturada e organizada. **Matplotlib** e **Seaborn** são ferramentas gráficas, utilizadas para plotar gráficos dos mais variados tipos. A seguir iremos ver um pouco mais destas bibliotecas em algumas aplicações representação dos dados.

4.1 Leitura de arquivos

A maioria dos conjuntos de dados disponibilizados online, seguem um padão de arquivo de texto chamado (*comma-separated values*), com extensão to tipo `.csv`. Nestes arquivos, a primeira linha é o cabeçalho, contendo o nome das colunas separadas por vírgulas, e as demais linhas são registros, com valores também separados por vírgulas, na mesma disposição da definição dos cabeçalhos.

Por este motivo, a função da biblioteca **Pandas** `read_csv()` é provavelmente a mais vista em kernels de análise de dados pela internet. Este comando lê um arquivo a partir de seu nome, analisando todos os valores de cada coluna e definindo o melhor tipo para aquele atributo, e este conjunto de valores é carregado como um objeto da classe *DataFrame*.

No exemplo abaixo, fazemos a leitura de um banco de dados com revisão de mais de 150.000 vinhos, obtido no website Kaggle [1]. Utilizamos a função `info()` para nos mostrar algumas informações básicas sobre o DataFrame que foi criado, que mostra ao usuário o nome da variáveis carregadas, quantos registros não-nulos estão disponíveis, qual o tipo foi reservado para cada variável (inteiro, *float* ou *string*), e qual o espaço em memória foi necessário no carregamento.

```
>> import pandas as pd
>> df = pd.read_csv('wine-reviews.csv')
>> df.info()


<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150930 entries, 0 to 150929
Data columns (total 10 columns):
country       150925 non-null object
description   150930 non-null object
designation   105195 non-null object
points        150930 non-null int64
price         137235 non-null float64
province      150925 non-null object
region_1      125870 non-null object
region_2      60953 non-null object
variety       150930 non-null object
winery        150930 non-null object
dtypes: float64(1), int64(1), object(8)
memory usage: 11.5+ MB
```

Uma outra forma de verificar o número de registros faltantes ou não preenchidos por linha, é utilizando a função `isna()`. No exemplo a seguir utilizamos também a função `sum()` pelo eixo 0, para somar o número de dados faltantes por coluna.

[1] Wine Reviews dataset - https://www.kaggle.com/zynicide/wine-reviews

```
>> df.isna().sum(axis=0)
country          5
description      0
designation  45735
points           0
price        13695
province         5
region_1     25060
region_2     89977
variety          0
winery           0
dtype: int64
```

Outra função também usada é `head()`, que imprime os 5 primeiros registros do conjunto de dados, ou `tail()`, que imprime os últimos 5 registros, ou `sample()`, que imprime 5 registros aleatórios. Caso o usuário queira menos ou mais do que 5 registros, basta passar para a função o argumento com um número inteiro.

```
>> df.sample(2)
```

country	designation	points	price	province	region_1	region_2	variety	winery
Italy	In Violas	92	40.0	Tuscany	Cortona	NaN	Merlot	Poliziano
Italy	Campo ...	93	NaN	Tuscany	Toscana	NaN	Red Blend	Campo ...

Cada atributo de um *DataFrame*, é uma série, que pode ser acessada a partir do nome da variável, passada como índice – entre colchetes. No exemplo a seguir utilizamos ainda o método `unique()` para selecionar todos os valores únicos da série 'country', que contempla os países com avaliações de vinhos disponíveis.

```
>> df['country'].unique()
array(['Albania', 'Argentina', 'Australia', 'Austria','Bosnia and Herzegovina'
    , 'Brazil', 'Bulgaria', 'Canada', 'Chile', 'China', 'Croatia', 'Cyprus', '
    Czech Republic', 'Egypt', 'England', 'France', 'Georgia', 'Germany', '
    Greece', 'Hungary', 'India', 'Israel', 'Italy', 'Japan', 'Lebanon', '
    Lithuania', 'Luxembourg', 'Macedonia', 'Mexico', 'Moldova', 'Montenegro',
    'Morocco', 'New Zealand', 'Portugal', 'Romania', 'Serbia', 'Slovakia', '
    Slovenia', 'South Africa', 'South Korea', 'Spain', 'Switzerland', 'Tunisia
    ', 'Turkey', 'US', 'US-France', 'Ukraine', 'Uruguay', nan], dtype=object)
```

Outro método bem útil, que nos permite uma rápida interpretação de variáveis numérica, é o `describe()`. Este método imprime algumas estatísticas descritivas básicas da amostra, como tamanho, média, desvio padrão, mínimo, máximo e quartis. A função `round()` também é mostrada no exemplo a seguir para que apenas 2 decimais fossem impressos.

```
>> df.describe().round(2)
```

	points	price
count	150930.00	137235.00
mean	87.89	33.13
std	3.22	36.32
min	80.00	4.00
25%	86.00	16.00
50%	88.00	24.00
75%	90.00	40.00
max	100.00	2300.00

Por fim apresentamos um último comando `groupby()` de agregação, que permite agrupar valores por alguma variável categórica. No exemplo abaixo iremos selecionar os 10 tipos de vinho com valor médio mais caro. Logo após o comando `groupby()`, note que precisamos utilizar uma função de agregação estatística – como média, mediana, contagem, mínimo, máximo, etc – ou alguma função específica criada pelo usuário.

```
>> df.groupby('variety').mean().sort_values('price', ascending=False).head(10)
    .round(2)
```

variety	points	price
Cabernet-Shiraz	96.00	150.00
Muscadel	92.90	141.30
Mazuelo	88.00	98.50
Tinto Fino	89.90	83.12
Mission	90.57	82.57
Tokay	93.29	82.11
Carignan-Syrah	92.00	80.00
Champagne Blend	90.00	78.62
Debit	86.66	72.33
Picolit	91.03	71.86

O *DataFrame* é o está para a biblioteca **Pandas**, assim como o *Array* está para a biblioteca **Numpy**. Fato interessante é que, o próprio DataFrame contém um *Array*, que pode ser obtido através da propriedade *values*.

4.2 Gráficos

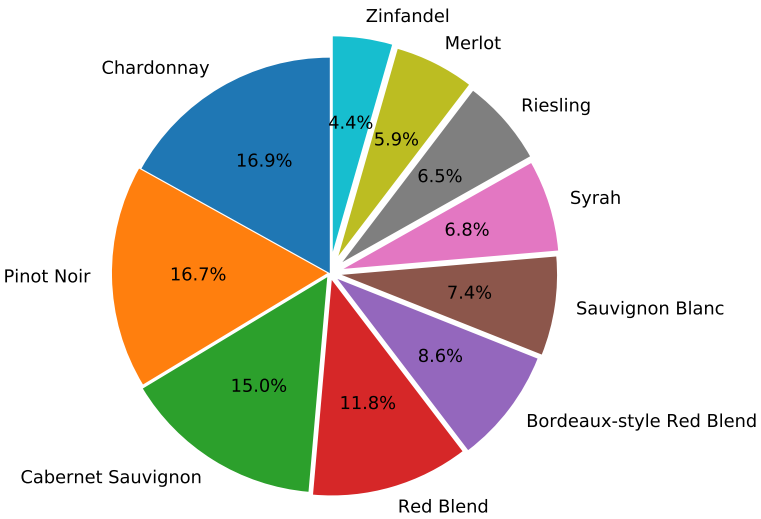
O **Matplotlib.PyPlot** é uma das bibliotecas fundamentais para geração de gráficos e manipulação de imagens em Python.

- *plt.plot()* - representação de gráficos de linhas, muito útil para representação de séries temporais, linhas de tendências, funções, dentre outras aplicações.

- *plt.scatter()* - representação de gráficos de dispersão, geralmente aplicado para avaliar visualmente o diagrama de dispersão de duas variáveis.
- *plt.bar()* - representação de gráficos em barra, pode ser utilizada tanto para dados categóricos quanto numéricos.
- *plt.hist()* - gera o histograma de frequência é a primeira ferramenta para compreendermos a distribuição de uma série de dados uni-variada.
- *plt.boxplot()* - forma de visualização estatística que permite compreender a dispersão dos dados para diferentes classes.
- *plt.pie()* - gráfico de pizza ou torta, utilizado para mostrar a representatividade das variáveis categóricas dentre o total de amostras.

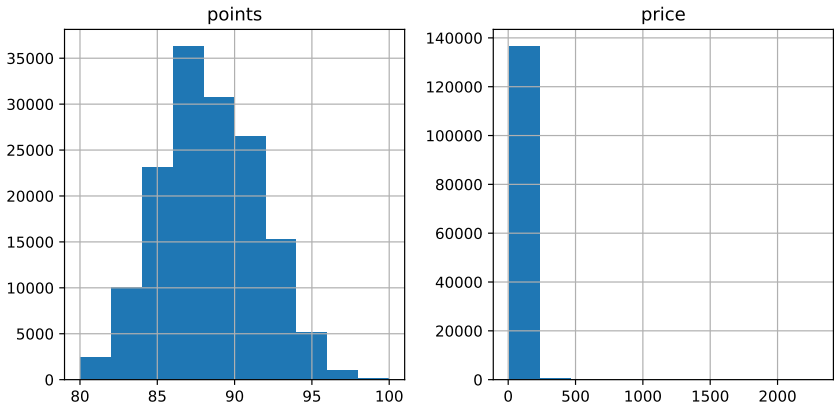
Mais informações e documentação sobre a biblioteca podem ser acessadas na página oficial do projeto em PyPlot.

```
>> x = pd.value_counts(df['variety'])
>> cat = x[:10].index.values
>> freq = x[:10].values
>>
>> np.append(cat, 'Others')
>> np.append(freq, np.sum(x[10:]))
>>
>> plt.figure(figsize=(6,6))
>> plt.pie(freq, labels=cat, explode=np.linspace(0,0.1,10), startangle=90,
    autopct='%1.1f%%')
>> plt.show()
```



A maioria das funções gráficas do **Matplotlib** estão também presentes e integradas ao quadro de dados do **Pandas**, como funções do objeto, que podem ser facilmente evocadas conforme exibido no exemplo abaixo.

```
>> df.hist(figsize=(8,4))
>> plt.show()
```



Um tutorial de visualização utilizando funções próprias do, pode ser encon-

trado na documentação de Visualização - Pandas.

Iremos agora dar exemplo de uma outra biblioteca de representação de dados mais avançada chamada de **Seaborn**, que funciona em colaboração com objetos do tipo *DataFrame*.

Esta biblioteca inúmeras funções para representações gráficas, que são agrupadas nas seguintes categorias.

1. Gráficos relacionais - permite a análise da relação entre duas variáveis, categóricas ou numéricas, através de gráficos de linhas e dispersão.

2. Gráficos categóricos - análise de variáveis categóricas utilizando gráficos de contagem, caixa, pontos, etc.

3. Gráficos de distribuição - histogramas e gráficos kde (função de densidade) para variáveis uni ou bi-dimensionais.

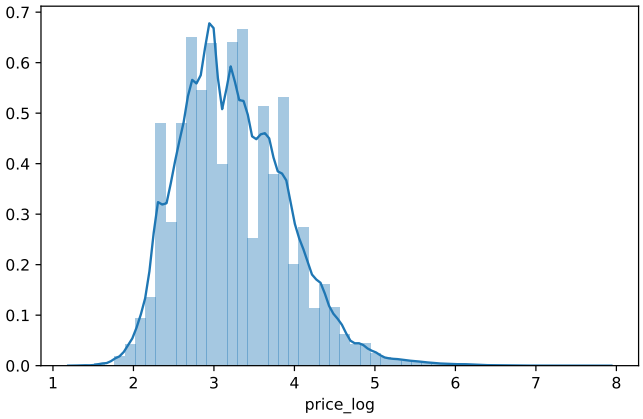
4. Gráficos de regressão - linhas de tendência a partir de modelos lineares, para estimar o comportamento conjunto de dados relacionados.

5. Gráficos de matrizes - mapas de calor para representação de grandezas ou funções e mapas de agrupamento.

No próximo exemplo iremos utilizar um gráfico de distribuição de uma variável plotada no exemplo anterior, no entanto transformada para a escala logarítmica.

Primeiramente iremos criar uma nova coluna, a partir do logaritmo do preço original. Usamos o o comando `distplot()`, que plota a distribuição da variável transformada, que inclui ambos histogramas e função de densidade. Selecionamos a série criada e utilizamos o método `dropna()` para eliminar os registros cujos dados sejam faltantes, para evitar erros na função de plotagem.

```
>> df['price_log'] = np.log(df['price']) # Cria nova coluna com o log do preco
>> sns.distplot(df['price_log'].dropna())
>> plt.show()
```



Ainda dentro do contexto de gráficos de distribuição, podemos fazer a análise da distribuição de duas variáveis simultaneamente, através da função `jointplot()`. Em seguida fazemos esta comparação usando a variável transformada do preço e das notas de avaliação. Observe o quanto é direto a criação de um gráfico sofisticado, a partir de poucas linhas de comando.

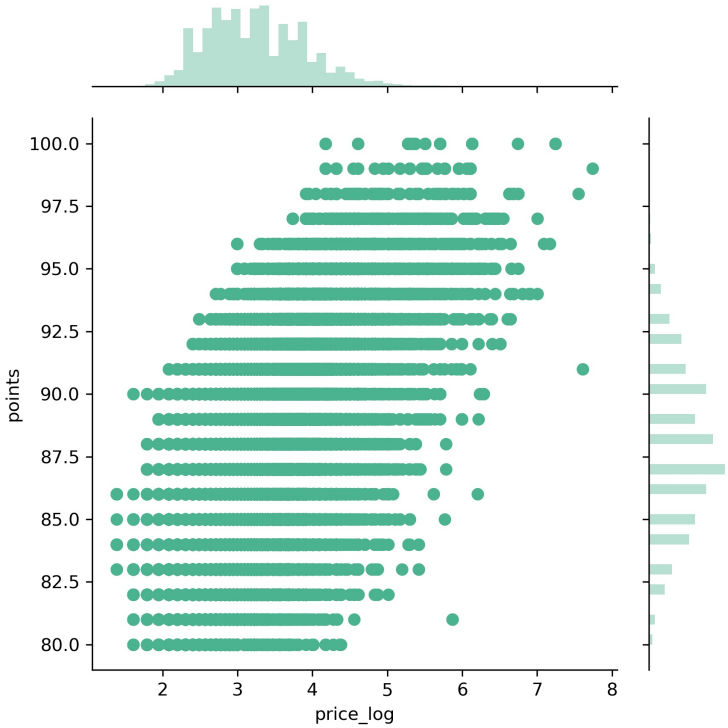
```
>> sns.jointplot('price_log', 'points', color="#4CB391", data=df)
>> plt.show()
```

A representação gráfica de variáveis categóricas é um processo que em geral requer bastante preparação dos dados, visto que em geral estes valores são do tipo *string*, e as bibliotecas mais básicas de plotagem requerem dados como valores numéricos. Este tipo de pré-processamento é realizado pelas funções de representação categórica do módulo, que contém diversas de funções para este tipo de gráfico.

A seguir damos o exemplo de duas funções de representação categórica, são elas o `stripplot()`, que plota as ocorrências dos dados por categoria dispersos pela variável preço, e o `pointplot()`, que irá nos retornar o preço médio dos vinhos de cada país.

```
>> plt.figure(figsize=(12,14))
>>
>> sns.stripplot(x="price", y="country",
>>             data=df, dodge=True, jitter=True,
>>             alpha=.25, zorder=1)
>>
>> sns.pointplot(x="price", y="country",
>>             data=df, dodge=.532, join=False, color='k',
>>             markers="|", scale=.75, ci=None)
>> plt.show()
```

Finalmente, os gráficos de caixa também podem ser aplicados aos dados utilizando-



se a função `boxplot()`. No nosso último exemplo deste capítulo, representamos a distribuição em caixas da variável nota por países.

```
>> plt.figure(figsize=(12,14))
>> sns.boxplot(x='points',y='country',data=df)
>> plt.show()
```

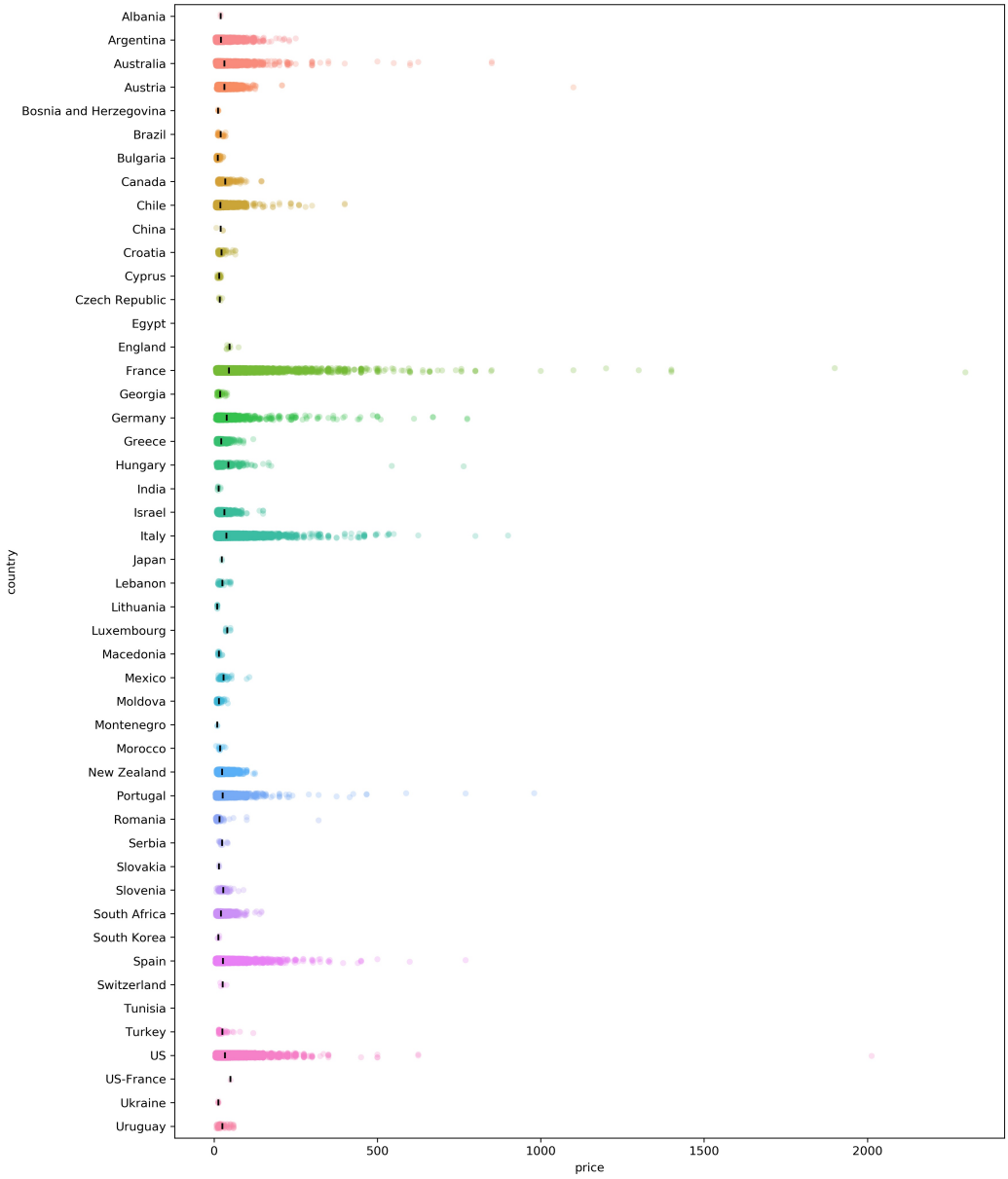
Para acessar a documentação do módulo e explorar mais funcionalidades, acesse o site do projeto.

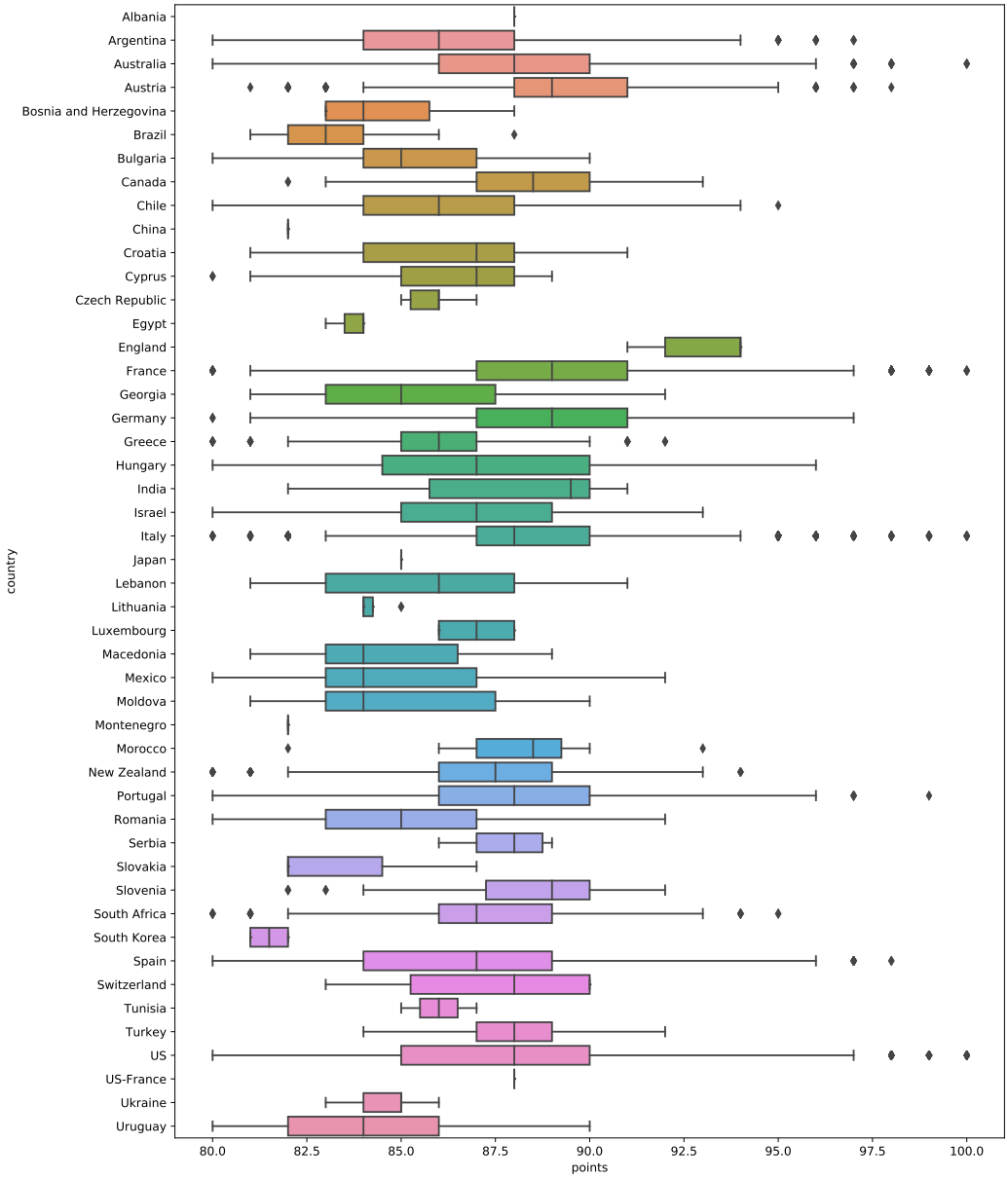
4.3 Exercícios

Exercício 1. Faça uma análise exploratória de um conjunto de dados do seu interesse, calculando as estatísticas descritivas básicas das variáveis mais importantes, e fazendo os gráficos que achar pertinente. Lembre-se que não existe fórmula ou resposta certa, o mais importante é se familiarizar-se com o conjunto de dados em questão.

Algumas fontes de dados públicos, incluindo repositórios populares e metaportais, para obtenção de dados para análise:

- Repositórios
  - UCI Machine Learning Repository
  - Kaggle datasets
  - Amazon's AWS datasets
- Metaportais
  - Portal Brasileiro de Dados Abertos
  - Data Portals
  - Open Data Monitor
- Outras fontes que listam portais de dados abertos
  - Quora.com question
  - Datasets subreddit


Albania
Argentina
Australia
Austria
Bosnia and Herzegovina
Brazil
Bulgaria
Canada
Chile
China
Croatia
Cyprus
Czech Republic
Egypt
England
France
Georgia
Germany
Greece
Hungary
India
Israel
Italy
Japan
Lebanon
Lithuania
Luxembourg
Macedonia
Mexico
Moldova
Montenegro
Morocco
New Zealand
Portugal
Romania
Serbia
Slovakia
Slovenia
South Africa
South Korea
Spain
Switzerland
Tunisia
Turkey
US
US-France
Ukraine
Uruguay
country
0
500
1000
1500
2000
price


Albania
Argentina
Australia
Austria
Bosnia and Herzegovina
Brazil
Bulgaria
Canada
Chile
China
Croatia
Cyprus
Czech Republic
Egypt
England
France
Georgia
Germany
Greece
Hungary
India
Israel
Italy
Japan
Lebanon
Lithuania
Luxembourg
Macedonia
Mexico
Moldova
Montenegro
Morocco
New Zealand
Portugal
Romania
Serbia
Slovakia
Slovenia
South Africa
South Korea
Spain
Switzerland
Tunisia
Turkey
US
US-France
Ukraine
Uruguay
country
80.0
82.5
85.0
87.5
90.0
92.5
95.0
97.5
100.0
points

Capítulo 5

Aprendizado supervisionado

5.1 Exemplo sintético

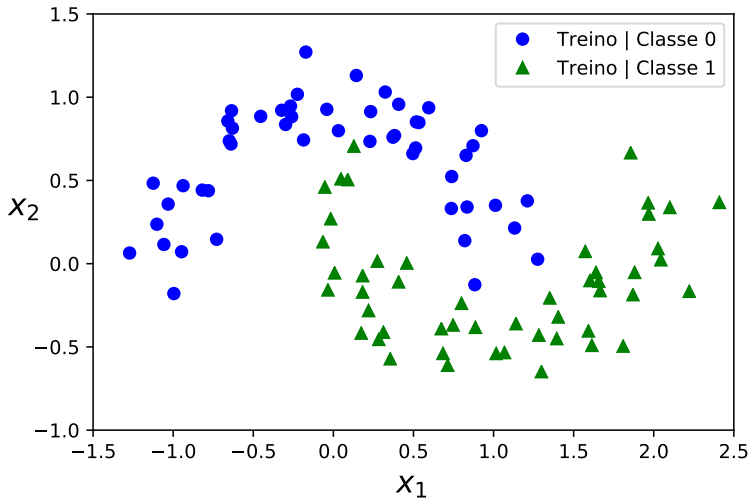
Neste primeiro exemplo, vamos criar um conjunto sintético de duas semi-círculos intercalados, cada qual representando uma classe diferente a ser prevista.

Nosso conjunto de dados é composto por apenas dois atributos x_1 e x_2, de forma que podemos visualizar em um plano bi-dimensional o problema criado. Queremos avaliar o desempenho de diferentes classificadores neste exemplo.

Para criar o iremos utilizar uma função do **scikit-learn** chamada `make_moons`, que cria um conjunto de dados de tamanho `n_samples` e ruído `noise` de acordo com o problema dado. O argumento `random_state` é um valor inteiro para que o gerador aleatório da função resulte sempre num mesmo resultado, também conhecido na computação como valor semente.

No trecho de código abaixo, definimos também a função `plot_dataset()`, capaz de plotar um gráfico do problema. Ela será bem útil para reusabilidade do código posteriormente.

```
>> from sklearn.datasets import make_moons
>> X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
>>
>> def plot_dataset(X, y, axes, marker=['bo','g^'], label='Treino'):
>>    plt.plot(X[:, 0][y==0], X[:, 1][y==0],
>>       marker[0], label=label + ' | Classe 0')
>>    plt.plot(X[:, 0][y==1], X[:, 1][y==1],
>>       marker[1], label=label + ' | Classe 1')
>>    plt.axis(axes)
>>    plt.xlabel(r"$x_1$", fontsize=16)
>>    plt.ylabel(r"$x_2$", fontsize=16, rotation=0)
>>    plt.legend()
>>
>> plot_dataset(X, y, [-1.5, 2.5, -1, 1.5]); plt.show()
```



Neste problema de classificação, buscamos um classificador que seja capaz de definir um limite físico entre os grupos, de forma que novas instâncias sejam classificadas com a maior acurácia possível.

Interessante notar que a partir desta imagem nós seres humanos seríamos capazes de facilmente, com a ajuda de um lápis, traçar uma curva que separe estas duas classes. Mas é importante lembrar que estamos lindando com um exemplo sintético, limitado a cem instâncias, e apenas dois atributos.

Em aplicações de aprendizado de máquina reais, é comum lidarmos com conjuntos de dados com milhões de instâncias e centena de milhares de atributos, por isso necessitamos de modelos computacionais satisfatórias para aplicarmos nestes casos. Outros fatores podem aumentar ainda mais a complexidade das aplicações: podem existir mais de duas classes, estas podem estar balanceadas, pode existir sobreposição de classes (instâncias de diferentes classes muito próximas ou dentro de um agrupamento de outra classe).

5.2 Separação dos conjuntos de treino e teste

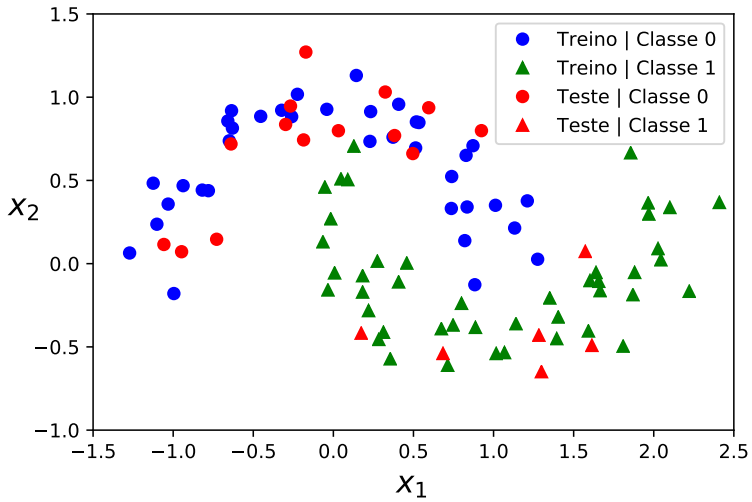
A primeira coisa que precisamos fazer para avaliar modelos de aprendizado supervisionado, é separar uma parte do conjunto de dados que temos para ser utilizada para **teste**. A parte que iremos utilizar para treinar e ajustar nossos modelos, é chamada de **treino**. Esta divisão é importante para conseguirmos avaliar o poder de generalização de nossos modelos.

Em bancos de dados públicos próprios para aprendizado de máquina, normalmente a própria pessoa a disponibilizar os dados, já faz a separação do conjunto de dados entre treino e teste. Assim é mais fácil para que pesquisadores e cientistas de dados avaliem seus modelos e comparem com os demais.

Há muita discussão a cerca do tamanho da amostra de dados que deve ser mantida para teste. Geralmente destina-se de 10% a 30% dos dados para teste, sendo esta decisão sendo dependente do tipo de problema e da dificuldade em se obter dados.

Para fazer a separação, uma função muito útil é o `train_test_split()`. Nela passamos os atributos X e as classes y, e qual o tamanho do conjunto de teste, e ela retorna os dois conjuntos de treino e teste, separados em X e y.

```
>> from sklearn.model_selection import train_test_split
>>
>> X_train, X_test, y_train, y_test = train_test_split(X, y,
>>                        test_size=0.2, random_state=42)
>>
>> plot_dataset(X_train, y_train, [-1.5, 2.5, -1, 1.5]);
>> plot_dataset(X_test, y_test, [-1.5, 2.5, -1, 1.5],
>>                marker=['ro','r^'], label='Teste');
>> plt.show()
```



Note que novamente utilizamos um valor fixo para semente com o argumento `random_state` da função. Isto acontece, pois no exemplo abaixo queremos obter a mesma separação aleatória sempre que executarmos o comando.

5.3 O modelo linear

O modelo matemático mais simples que podemos idealizar para realizar a separação das classes em um plano de duas dimensões é uma reta. O modelo mais

próximo desta interpretação é a **regressão logística**, cujo ajuste busca determinar os coeficientes de uma função logística que funcionam como um separador binário, dado um valor limítrofe para cada atributo.

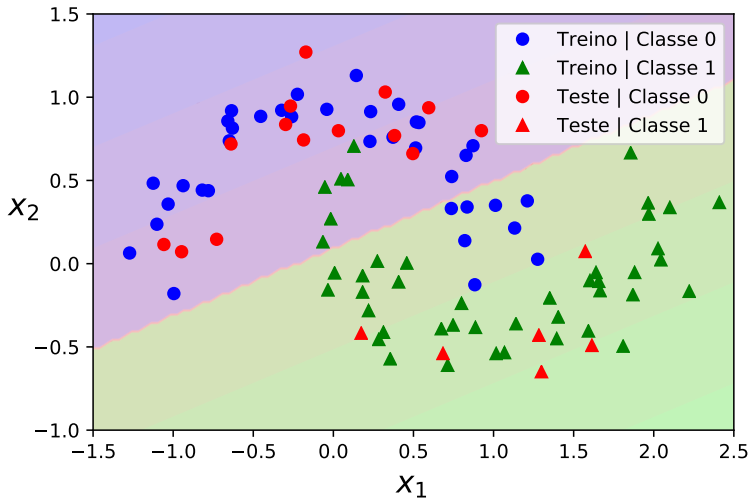
Por hora, não entraremos na formulação matemática da regressão logística ou de qualquer um dos outros estimadores utilizados, pois nosso interesse maior é na compreensão das saídas dos modelos em si.

Definimos a função `plot_predictions()` a seguir, que é capaz de representar graficamente os contornos da função de um dado classificador ajustado.

```
>> def plot_predictions(clf, axes):
>>     x0s = np.linspace(axes[0], axes[1], 100)
>>     x1s = np.linspace(axes[2], axes[3], 100)
>>     x0, x1 = np.meshgrid(x0s, x1s)
>>     X_ = np.c_[x0.ravel(), x1.ravel()]
>>     y_pred = clf.predict(X_).reshape(x0.shape)
>>     if hasattr(clf,'decision_function'):
>>         y_decision = clf.decision_function(X_).reshape(x0.shape)
>>     else:
>>         y_decision = clf.predict(X_).reshape(x0.shape)
>>
>>     plt.contourf(x0, x1, y_pred, cmap=plt.cm.brg, alpha=0.2)
>>     plt.contourf(x0, x1, y_decision, cmap=plt.cm.brg, alpha=0.1)
```

Da biblioteca de modelos linear do **scikit-learn**, importamos a regressão linear e ajustamos o modelo ao nosso conjunto de treino. Usamos as funções gráficas definidas anteriormente para desenhar o conjunto de teste e o limite das funções gerados.

```
>> from sklearn.linear_model import LogisticRegression
>> clf = LogisticRegression()
>> clf.fit(X_train, y_train)
>>
>> plot_predictions(clf, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_train, y_train, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_test, y_test, [-1.5, 2.5, -1, 1.5],marker=['ro','r^'], label
    ='Teste')
```



Como podemos analisar do gráfico gerado, o modelo define uma reta, que separa o plano em duas regiões. Todos as instâncias abaixo da reta seriam classificados como pertencentes a classe 1, enquanto todas as instâncias acima da reta serão classificados como pertencentes a classe 1.

Embora tenhamos obtido um resultado satisfatório com este classificador, é possível verificar que muitas das instâncias da classe 0 estão ocupando a região definida para a classe 1, e vice-versa.

5.4 Extendendo o modelo linear

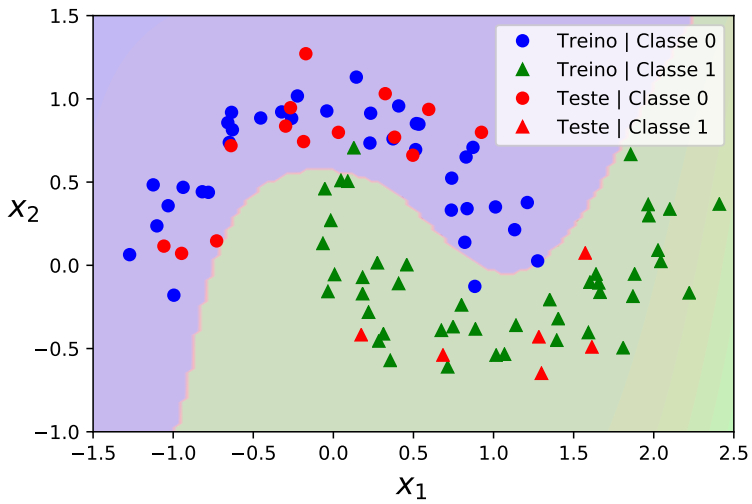
Uma forma simples de melhorarmos a acurácia do nosso classificador linear, é realizar a transformação polinomial dos dois atributos que temos, obtendo novos atributos calculados que serão utilizados pelo modelo linear. Para isso usamos uma função de pré-processamento da biblioteca chamada `PolynomialFeatures()`.

```
>> from sklearn.preprocessing import PolynomialFeatures
>> X_poly = PolynomialFeatures(degree=5).fit_transform(X)
>>
>> print("N atributos antes polinomial: %i" % X.shape[1])
>> print("N atributos depois polinomial: %i" % X_poly.shape[1])
N atributos antes polinomial: 2
N atributos depois polinomial: 21
```

Neste exemplo utilizamos o grau de polinômio igual a 5, o que nos retornou um conjunto de treino com 21 atributos ao invés de 2.

Iremos utilizar um **pipeline** para realizar a transformação polinomial e depois ajustar o modelo logístico. Não se preocupe com o conceito de agora, pois iremos tratar melhor este recurso nos próximos capítulos.

```
>> from sklearn.pipeline import Pipeline
>> from sklearn.preprocessing import PolynomialFeatures
>>
>> clf = Pipeline([
>>     ("poly_features", PolynomialFeatures(degree=5)),
>>     ("logistic", LogisticRegression())
>>     ])
>>
>> clf.fit(X_train, y_train)
>>
>> plot_predictions(clf, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_train, y_train, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_test, y_test, [-1.5, 2.5, -1, 1.5],
>>                     marker=['ro','r^'], label='Teste')
```

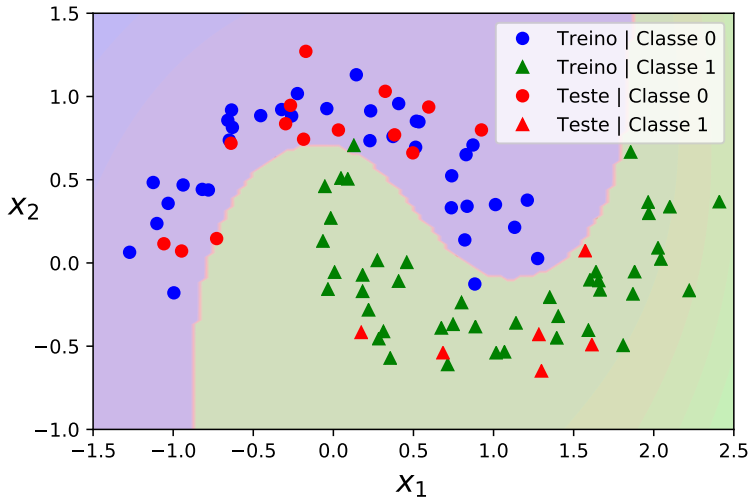


Verifique a curva formada pelo classificador para gerar os planos. Esta não é mais uma reta, e sim uma curva de um polinômio de grau menor ou igual a cinco. Comparado ao modelo anterior, as áreas destinadas para cada classe parecem mais apropriadas para descrever o problema.

5.5 Testando outros classificadores

Nos próximos códigos iremos repetir o mesmo processo de ajuste de modelos que realizamos anteriormente para outros algoritmos de classificação disponíveis no pacote. O exemplo a seguir é a **máquina vetor suporte**, uma meta-heurística que busca traçar um plano separador ou hiper-plano. No exemplo abaixo é definido uma função polinomial de grau 5 como núcleo do classificador.

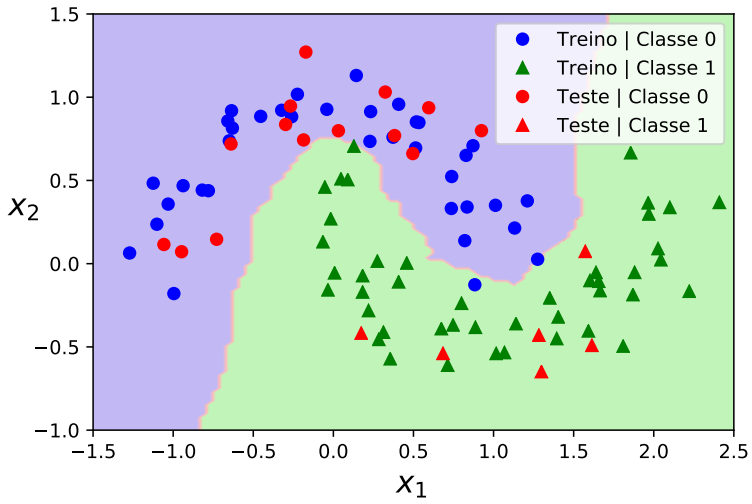
```
>> from sklearn.svm import SVC
>> clf = SVC(kernel="poly", degree=5, coef0=1, C=1., gamma='auto')
>> clf.fit(X_train, y_train)
>>
>> plot_predictions(clf, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_train, y_train, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_test, y_test, [-1.5, 2.5, -1, 1.5],
>>                    marker=['ro','r^'], label='Teste')
```



O resultado é de certa forma bem próximo ao exemplo dado anterior. Isto é esperado, visto que ambos utilizam a mesma transformação polinomial de grau 5.

Iremos ajustar agora um modelo tipicamente usado para clusterização em aprendizado não-supervisionado, o **k-vizinhos mais próximos**. Este modelo classifica uma nova amostra dada sua proximidade com as k instâncias mais próximos.

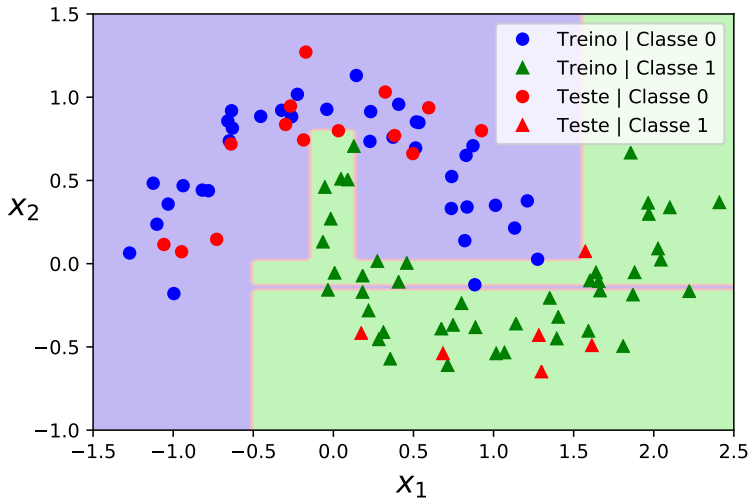
```
>> from sklearn.neighbors import KNeighborsClassifier
>> clf = KNeighborsClassifier(n_neighbors=5)
>> clf.fit(X_train, y_train)
>>
>> plot_predictions(clf, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_train, y_train, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_test, y_test, [-1.5, 2.5, -1, 1.5],
>>                    marker=['ro','r^'], label='Teste')
```



A fronteira gerada é parecida com a traçada nos exemplos anteriores, no entanto a função de separação não é contínua. Isto ocorre pois o modelo não é linear, sendo estritamente relacionado aos dados disponíveis no conjunto de treino.

Outra família de modelos muito conhecida e aplicada no domínio de conhecimento de aprendizado de máquina, é a **árvore de decisão**. Este modelo particiona os dados iterativamente a partir de um atributo por vez, a partir de um critério de pureza (normalmente ganho de informação ou entropia).

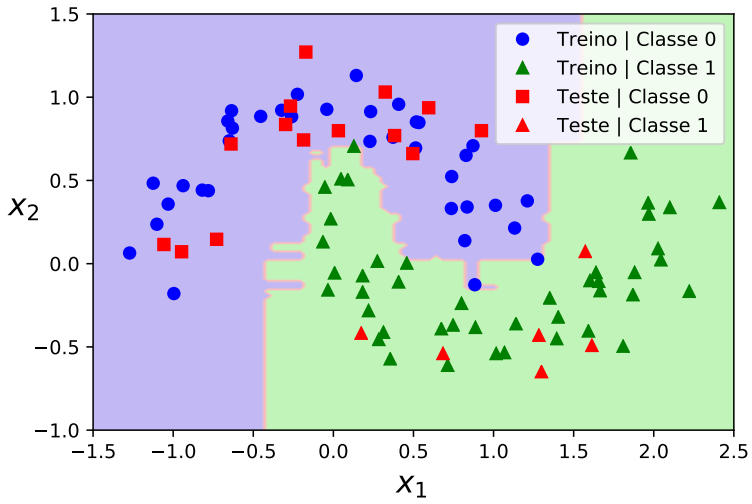
```
>> from sklearn.tree import DecisionTreeClassifier
>> clf = DecisionTreeClassifier(random_state=32)
>> clf.fit(X_train, y_train)
>>
>> plot_predictions(clf, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_train, y_train, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_test, y_test, [-1.5, 2.5, -1, 1.5],
>>                    marker=['ro','r^'], label='Teste')
```



É possível verificar que a curva formada por este último modelo é bem diferente dos anteriores, como se diversas retas houvessem sido traçadas para delimitar a área de cada uma das classes.

A **floresta randômica** é um dos modelos mais poderosos, desenvolvido a partir da árvore de decisão. Trata-se de um conjunto (comitê de classificação) de árvores de decisão ajustados usando diferentes porções de atributos e instâncias. A previsão da classe é baseada na combinação da previsão de cada árvore individual (voto ou média da saída de probabilidade).

```
>> from sklearn.ensemble import RandomForestClassifier
>> clf = RandomForestClassifier(n_estimators=30)
>> clf.fit(X_train, y_train)
>>
>> plot_predictions(clf, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_train, y_train, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_test, y_test, [-1.5, 2.5, -1, 1.5],
>>                    marker=['ro','r^'], label='Teste')
```

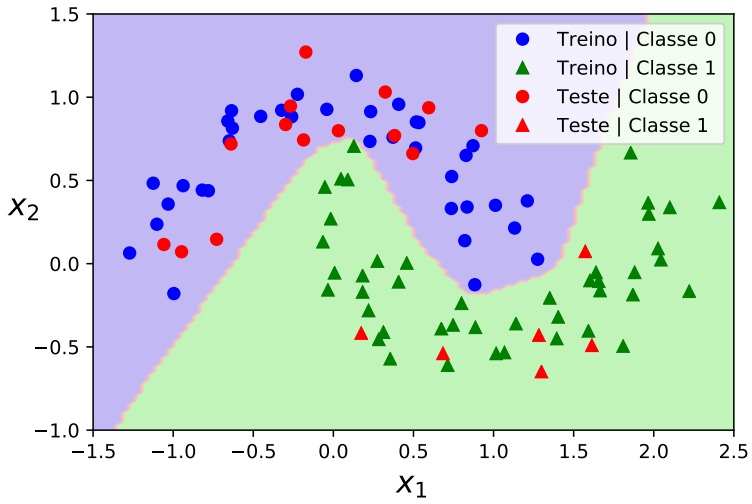


A floresta randômica gera uma função ainda mais complexa para separação das classes. Embora seja um modelo muito poderoso, este modelo requer cautela quando for ajustado para que não ocorra *overfitting* nos dados.

Sobreajuste ou *overfitting*, ocorre quando um modelo se especializa excessivamente no conjunto de dados de treino, obtendo baixos erros neste conjunto, mas quando aplicado a um novo conjunto de teste, seu desempenho não é próximo ou satisfatório do esperado.
De forma análoga, **subajuste** ou *underfitting*, ocorre quando o modelo é muito simples para descrever o fenômeno desenvolvido.

O último modelo que iremos ajustar é o **perceptron multi-camadas**, uma rede neural com três camadas, na qual todos os neurônios são interconectados. É um modelo de alta complexidade, com grande capacidade preditiva mas também sujeito sobreajuste, assim como a floresta randômica.

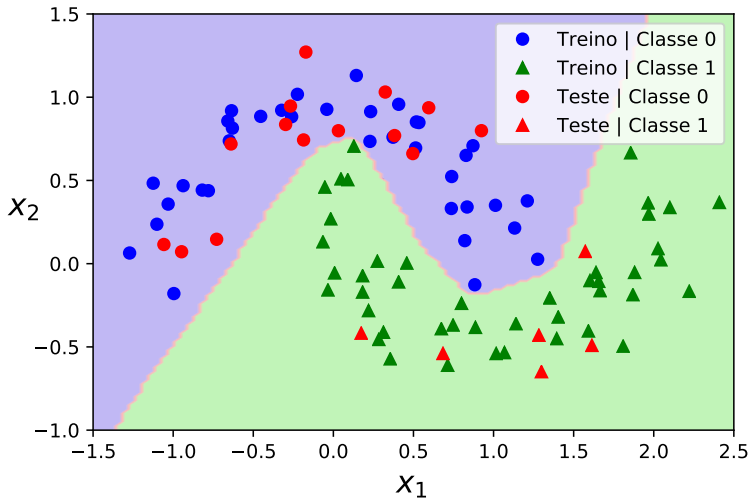
```
>> from sklearn.neural_network import MLPClassifier
>> clf = MLPClassifier(hidden_layer_sizes=(30,20,10),
>>                     random_state=42, max_iter=2000)
>> clf.fit(X_train, y_train)
>>
>> plot_predictions(clf, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_train, y_train, [-1.5, 2.5, -1, 1.5])
>> plot_dataset(X_test, y_test, [-1.5, 2.5, -1, 1.5],
>>                     marker=['ro','r^'], label='Teste')
```



A curva gerada apresenta um padrão próximo ao observado nos modelos de máquina suporte polinomial, no entanto a fronteira se expande de forma a incluir todas as instâncias de treinamento. Observe também que existe certa não-linearidade nas curvas geradas, quando comparado com o modelo polinomial.

5.6 Ajuste de parâmetros

```
>> from sklearn.svm import SVC
>>
>> for degree in [1,2,3,5]:
>>     clf = SVC(kernel="poly", degree=degree, coef0=1,
>>                 C=1, gamma='auto')
>>     clf.fit(X_train, y_train)
>>     plt.figure(figsize=(4,3))
>>     plot_predictions(clf, [-1.5, 2.5, -1, 1.5])
>>     plot_dataset(X_train, y_train, [-1.5, 2.5, -1, 1.5])
>>     plot_dataset(X_test, y_test, [-1.5, 2.5, -1, 1.5],
>>                         marker=['rs','r^'], label='Teste')
>>     plt.title("degree = "+str(degree)); plt.show()
```



5.7 Métricas de avaliação de modelos

Como podemos avaliar qual modelo é mais eficiente para um dado problema?

Para avaliarmos o desempenho dos modelos, precisamos definir uma ou mais **métricas** de avaliação. Em problemas de classificação, o mais comum é a **acurácia**, que é a taxa de classificações correta das classes, enquanto em problemas de regressão, o mais comum é o coeficiente de determinação $R2$. A seguir, listamos algumas outras métricas conhecidas para problemas de:

- **Classificação**: precisão e recordação, duas métricas concorrentes referentes a taxa de classificação correta da classe minoritária (menos comum) e a abrangência do teste; F1 score, média ponderada entre precisão e recordação; coeficiente de concordância de kappa, calculado a partir da matriz de confusão; área sob a curva de característica do operador-receptor (AUC ROC), usada em problemas de classe desbalanceados; custo total, calculado quando se sabe o custo do erro de classificação errada de cada uma das classes.

- Regressão: erro quadrático médio; errro percentual absoluto médio; erro absoluto médio, vantagem de estar na mesma escala da variável prevista, no entanto a função não é contínua 9;logaritmo do erro quadrático médio; erro quadrático mediano, apresenta maior robustez a *outliers*; índice de regressão da variância explicada.

A seguir, iremos avaliar a acurácia dos estimadores que apresentamos anteriormente, para o problema sintético das duas semi-luas. A métrica utilizada é a

precisão, dada pelo número de Verdadeiros Positivos / (Verdadeiros Positivos + Falso Positivos).

```
>> from sklearn.metrics import precision_score, classification_report
>> clf_list=[
>>    ('LOG', LogisticRegression(),
>>    ('SVC', SVC(kernel="poly", degree=5, coef0=1, C=1, gamma='auto')),
>>    ('KNN', KNeighborsClassifier(n_neighbors=5)),
>>    ('CART', DecisionTreeClassifier()),
>>    ('RF', RandomForestClassifier(n_estimators=30)),
>>    ('MLP',MLPClassifier(hidden_layer_sizes=(30,20,10),random_state=42,
    max_iter=2000)),
>>  ]
>>
>> X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
>> X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                 random_state=42)
>>
>> print("Modelo\tTreino\tTeste")
>> for name, clf in clf_list:
>>    clf.fit(X_train, y_train)
>>    print("{}\t{:.4f}\t{:.4f}".format(name, clf.score(X_train, y_train), >>
     precision_score(y_test, clf.predict(X_test))))

Modelo Treino Teste
LOG    0.8571 0.7692
SVC    0.9857 0.8333
KNN    0.9857 0.8333
CART   1.0000 0.8333
RF     1.0000 0.8333
MLP    1.0000 0.7692
```

A partir dos resultados verificamos que todos estimadores obtiveram a mesma precisão, com exceção do modelo logístico (LOGIT) e do perceptron multi-camadas (MLP). Neste caso houve empate pois são o problema é composto por poucos pontos, e sua complexidade é relativamente baixa.

Ainda assim, comparando o resultado com a análise gráfica que fizemos de cada estimadores, podemos ver que a regressão logística era um modelo menos complexo do que o problema, ajustando apenas um reta, e assim não sendo suficiente para delimitar as classes.

O desempenho do perceptron multi-camadas foi muito alto no conjunto de treino, classificando corretamente todas as instâncias, porém ao ser aplicado a um conjunto de dados novo, seu desempenho foi bem menor. Isto ocorre devido o fenômeno de sobreajuste, ou *overfitting*, no caso o modelo aplicado apresenta maior complexidade do que o necessário para a resolução do problema.

Embora a floresta randômica tenha obtido mesmo valor do que os demais mo-

delos, analisando visualmente os modelos anteriores, verificamos que a área delimitada pelo modelo apresenta uma complexidade acima do esperado para o problema.

5.8 Exercícios

Os seguintes exercícios são propostos para fixação de conhecimento deste capítulo.

Exercício 1. Ajuste um modelo linear e um modelo não-linear à sua escolha ao conjunto de dados do SP500, disponível no GitHub da disciplina.

1. Calcule o erro médio quadrático de cada modelo no conjunto de teste.
2. Plote suas predições e o valor real observado, qual modelo você julga mais indicado e porquê?
3. Teste alguns parâmetros para os modelos e verifique o que muda.

Exercício 2. Crie um modelo que seja capaz de reconstruir a máscara do conjunto de dados 'mask.csv', disponível no github, usando apenas o conjunto de dados de treino como entrada.

1. Teste pelo menos 3 métodos diferentes (regressão logística, KNN, SVM, rede neural,...) e compare com o desenho original. Qual modelo você escolheria?
2. Calcule a acurácia do modelo para te ajudar na decisão.

Exercício 3. Conjunto de dados Iris.

1. Ajuste um modelo de classificação ao conjunto de dados iris.
2. Utilize a função sklearn.metrics.classification_report para avaliar seu modelo.
3. Analise e explique a saída do relatório.
4. Teste modelos e parâmetros diferentes

Capítulo 6

Aprendizado não-supervisionado

Uma outra abordagem muito comum em *machine learning*, é o aprendizado não-supervisionado, também conhecido como clusterização ou agrupamento. Neste tipo de problema, não existe uma variável de interesse que queremos prever ou classificar, e busca-se separar todas as observações que tempos em grupos com maior similaridade entre si. Para isso, as técnicas mais comumente aplicadas são baseadas na distância euclideana entre as informações que temos sobre os dados.

Imagine que temos o seguinte problema. Em um banco de dados de uma empresa, existe pelo menos dois tipos de clientes: varejo e atacado. Clientes a varejo realizam compras com menor frequência, pagam valores mais alto como preço unitário, o valor total gasto anualmente é relativamente baixo, compram uma unidade de cada produto. Por outro lado, os clientes atacado realizam grandes compras, com frequência quinzenal ou mensal, possuem um gasto anual bem alto, compram muitas unidades de cada produto e logo pagam menos do que o preço unitário registrado.

Suponha ainda que não sabemos previamente (não existem rótulos) qual cliente é varejo e atacado. Faria sentido neste caso, a aplicação de um modelo de agrupamento. A maioria dos modelos requer a definição do número de grupos, que neste caso sabemos que é dois.

No entantSobreajute o, o modelo poderia ser estendido e criado mais grupos. Por exemplo, pode ser que existam clientes a varejo que só comprem com desconto/ofertas, outros que façam compras recorrente toda semana. Assim, determinar estes grupos e traçar uma estratégia para atender cada tipo de cliente, pode trazer um grande ganho para a empresa deste exemplo. Por isso a aplicação deste tipo de técnica é tão importante e imprescindível em aprendizado de máquina.

Finalmente, existem casos em que estas técnicas podem ser combinadas com as de aprendizado supervisionado. Isso é: existem alguns dados que são rotulados, enquanto a maioria não é. Na prática: os especialistas da empresa conseguiram rotular alguns dados, que eles têm certeza que são clientes varejo e alguns que

são atacado. Dessa forma conseguimos rotular o restante dos dados a partir de sua proximidade às instâncias conhecidas. Esta área é conhecida como aprendizado semi-supervisionado, mas não entraremos em detalhe neste livro e ilustramos apenas a título de conhecimento geral.

6.1 Análise de Componentes Principais

Os métodos de decomposição são muito populares na área de ciência dos dados, pois são capazes de diminuir a dimensão de entrada de um problema, selecionando aqueles atributos que mais descrevem a variância dos dados. Desta forma, conseguimos ajustar modelos de aprendizado com uma matriz de treino reduzida, diminuindo o tempo de treinamento de outros modelos supervisionados. Por serem normalmente combinados a outros modelos de classificação, estes métodos também são conhecidos como técnicas de pré-processamento por alguns autores.

Dentre estes métodos a análise de componentes principais (PCA), é o método mais conhecido. Vamos mostrar a seguir como implementar o método em Python utilizando a biblioteca **scikit-learn.decomposition**.

Vamos começar importando algumas bibliotecas básicas já conhecidas, e definindo algumas funções para representação dos dados. A função **PCA** é importada conforme o código a seguir. Definimos uma função simples **draw_vector** para desenhar dois vetores.

```
>> import pandas as pd
>> import numpy as np
>> import matplotlib.pyplot as plt
>> from matplotlib.colors import LogNorm
>> from mpl_toolkits.mplot3d import Axes3D
>> from sklearn import datasets
>> from sklearn.decomposition import PCA
>> from sklearn.mixture import GaussianMixture

>> def draw_vector(v0, v1, ax=None):
>>    ax = ax or plt.gca()
>>    arrowprops=dict(arrowstyle='->',
                      linewidth=2,
                      shrinkA=0, shrinkB=0)
>>    ax.annotate('', v1, v0, arrowprops=arrowprops)
```

Definimos um conjunto de dados aleatório de dimensão 2x200 (dois atributos, 200 observações), apenas para dar uma intuição de como o método de decomposição PCA funciona. Não se preocupe com o código agora, mas com as representações gráficas geradas na figura abaixo.

Verifique que o nosso conjunto aleatório está distribuído em suas duas dis-

tribuições. Olhando graficamente, a pergunta que fazemos é: quais seriam as duas direções nas quais nossos dados estão mais distribuídos? Em outras palavras, **quais vetores posso definir no meu espaço, de forma a descrever a maior variância dos dados**?

Por serem apenas duas dimensões, podemos verificar visualmente que melhor representa a variância dos conjuntos é o vetor $(0, 0) \rightarrow (-2, -1)$. Se quiséssemos analisar a distribuição dos dados em apenas uma dimensão (reduzindo de 2 para 1), avaliar os dados na direção do vetor mostrado no gráfico faria mais sentido do que no eixo x ou y plotado inicialmente. Pelo menos essa é a intuição que este método busca passar.

Note que o segundo vetor que é mostrado no gráfico é ortogonal (forma 90 graus) ao primeiro vetor. Esta é outra propriedade do método PCA, é que todos os vetores calculados **devem ser ortogonais**. A terceira propriedade é de que **o número máximo de vetores** (ou como formalmente chamado no método, componentes principais) **é igual à dimensão dos dados** - no caso desse exemplo, dois.

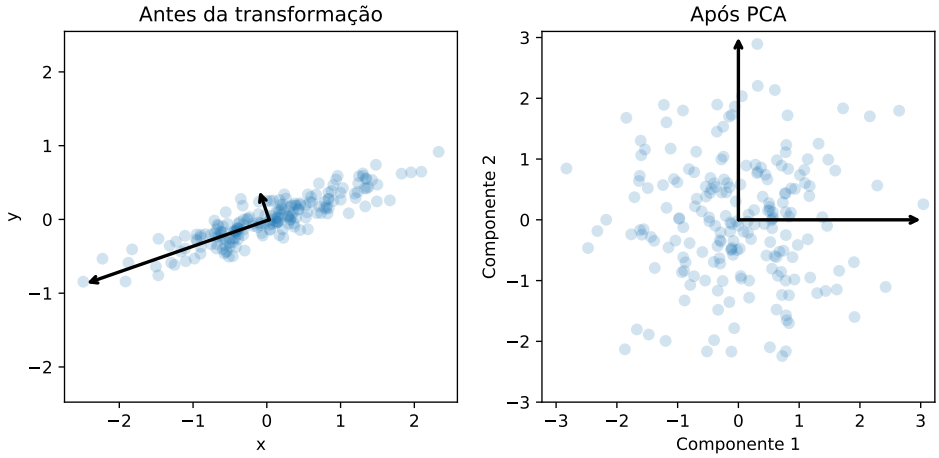
```
>> rng = np.random.RandomState(1)
>> X = np.dot(rng.rand(2, 2), rng.randn(2, 200)).T
>> pca = PCA(n_components=2, whiten=True)
>> pca.fit(X)

>> fig, ax = plt.subplots(1, 2, figsize=(12, 5))
>> fig.subplots_adjust(left=0.0625, right=0.95, wspace=0.1)

>> # plot data
>> ax[0].scatter(X[:, 0], X[:, 1], alpha=0.2)
>> for length, vector in zip(pca.explained_variance_, pca.components_):
      v = vector * 3 * np.sqrt(length)
      draw_vector(pca.mean_, pca.mean_ + v, ax=ax[0])
>> ax[0].axis('equal');
>> ax[0].set(xlabel='x', ylabel='y', title='Antes da transformação')

>> # plot principal components
>> X_pca = pca.transform(X)
>> ax[1].scatter(X_pca[:, 0], X_pca[:, 1], alpha=0.2)
>> draw_vector([0, 0], [0, 3], ax=ax[1])
>> draw_vector([0, 0], [3, 0], ax=ax[1])
>> ax[1].axis('equal')
>> ax[1].set(xlabel='Componente 1', ylabel='Componente 2',
         title='Após PCA',
         xlim=(-5, 5), ylim=(-3, 3.1))
```

O PCA é um transformador: depois de calculados os componentes principais, os dados são normalizados por estes vetores. No exemplo acima o conjunto transformado aparece distribuído uniformemente entre (-3,3) no novo espaço R^2 definido.



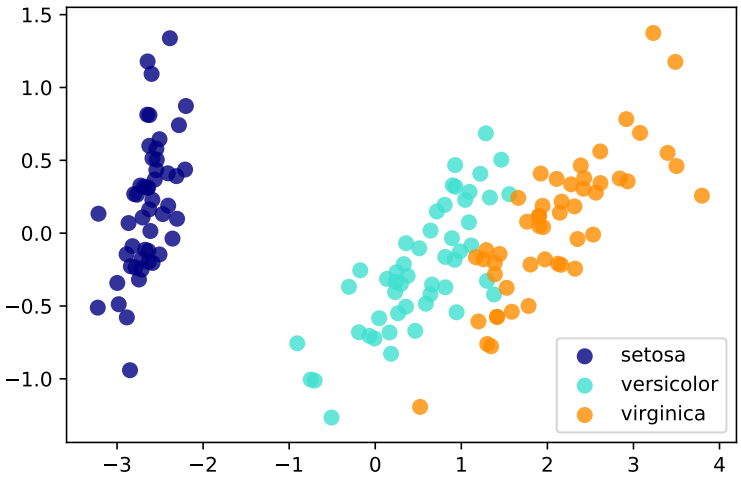
Vamos fazer o mesmo exercício, porém agora utilizando as bases de dado do conjunto **iris**. Queremos verificar dentre os atributos, quais são os principais componentes que descrevem a maior variância de dados.

Por esse conjunto de dados possuir mais de 2 atributos, para verificar a interação entre seus atributos numéricos, precisaríamos fazer uma representação par a par. Uma das vantagens do PCA, é que fazendo a representação do conjunto transformado a partir das duas componentes principais, conseguimos fazer um gráfico de duas dimensões com nossos dados. Adicionamos ainda o rótulo das espécies nos dados.

```
>> iris = datasets.load_iris()
>> X, y, target_names = iris["data"], iris["target"], iris['target_names']


>> pca = PCA(n_components = 2)
>> X2D = pca.fit_transform(X)
>> pd.DataFrame(data=X2D).head(8)
>> plt.figure()
>> colors = ['navy', 'turquoise', 'darkorange']
>> lw = 2
>> target_names = iris['target_names']
>> for color, i, target_name in zip(colors, [0, 1, 2], target_names):
      plt.scatter(X2D[y == i, 0], X2D[y == i, 1], color=color,
                  alpha=.8, lw=lw,label=target_name)
>> plt.legend(loc='best', shadow=False, scatterpoints=1)
>> plt.savefig('pca-iris.pdf',format='pdf',bbox_inches='tight')
>> plt.show()
```

É interessante notar como a partir de duas componentes principais, somos capazes de identificar visualmente ”fronteiras”entre as classes que geram os dados. E é justamente para isso que normalmente são aplicados o PCA e outros transfor-



madores de redução de dimensionalidade, eles são capazes de diminuir a complexidade do problema para aplicarmos algoritmos de classificação complexos.

Imagine uma base que tenham milhares, ou centenas de milhares de atributos. Em geral . Valeria a pena tentar ajustar um modelo complexo como uma rede neural, ou uma máquina de vetor suporte? Ou será que valeria a pena diminuir essa dimensão para 10 ou talvez 100 dos atributos que melhor descrevem os dados?

Para definir quantos componentes principais serão utilizados, o que em geral fazemos é verificar qual a variância explicada pelo número de componentes selecionados. Uma boa regra é adotar um número que represente pelo menos 90% da variabilidade da base, mas esse valor é puramente arbitrário e vai depender de sua aplicação.

Sobre os Autores

Rodrigo Barbosa de Santis é Doutor em Engenharia de Produção pela Universidade Federal de Minas Gerais. Possui experiência em gestão de operações e machine learning, tendo atuado como cientista de dados na Uber e Falconi, e atou em projetos como consultor nas empresas Natura, BASF, Johnson & Johnson e B2W. Suas habilidades incluem o desenvolvimento e implementação de sistemas preditivos e de suporte à decisão.
Lattes: http://lattes.cnpq.br/7356403238727506

Tiago Silveira Gontijo é Doutor em Engenharia de Produção pela UFMG na linha de pesquisa "Modelagem Estocástica e Simulação". Participou do Programa de Doutorado Acadêmico para Inovação (DAI) do CNPq. Mestre e Bacharel em Engenharia de Produção. Bacharel em Administração e em Economia. Possui 10 anos de experiência docente e em gestão educacional. Desenvolveu atividades docentes nas instituições: UFOP, UFSJ, PUC MG, UNIBH, UNA e Izabela Hendrix.
Lattes: http://lattes.cnpq.br/2865658247618316

MACHINE LEARNING E PYTHON

Um guia prático à ciência de dados

O aprendizado de máquina e Python estão mudando o mundo e agora é a sua chance de se juntar à revolução da ciência de dados! Este livro é a sua introdução completa à implementação do aprendizado de máquina com Python.

Com exemplos práticos e exercícios, este livro apresenta os conceitos fundamentais do aprendizado de máquina e como implementá-los em Python. Você vai descobrir como criar modelos precisos para previsões e como usar a análise de dados para obter insights valiosos.

Não importa se você é novo em aprendizado de máquina ou se já tem experiência, este livro é uma leitura essencial para todos que desejam se aprofundar em uma das áreas mais promissoras da ciência de dados. Prepare-se para entrar em um mundo novo e emocionante de descobertas e inovações com o aprendizado de máquina e Python!